NSCT Preparation

File 03: Data Structures & Algorithms (10%)

This file covers fundamental and advanced DSA concepts. It focuses on Big-O analysis, memory layout, and algorithmic strategy. Each question shows the correct answer in **bold**, followed by the explanation immediately under it.

Master Table of Contents (File 03 - Target: 240 MCQs)

#	Subtopic	Coverage	MCQs
01	Foundations of Data Structure and Algorithms	Basic	15
02	Linear Data Structures	Extended	25
03	Non-Linear Data Structures	Standard	18
04	Searching Algorithms	Standard	15
05	Sorting Algorithms	Extended	20
06	Hashing	Standard	15
07	Tree Algorithms	Extended	25
08	Graph Algorithms	Extended	25
09	Algorithm Design Techniques	Extended	25
10	Advanced Data Structures	Standard	15
11	String Algorithms	Standard	20
12	Complexity & Optimization	Standard	20
Total MCQs (File 03)			**243**

Subtopic 01: Foundations of Data Structure and Algorithms (15 MCQs)

Quick Notes

- **Data Structure:** A specialized format for organizing, processing, retrieving, and storing data in memory or on disk.
- **Big-O (O):** Upper bound. Represents the worst-case growth rate.
- **Big-Omega (Ω):** Lower bound. Represents the best-case growth rate.
- **Big-Theta (Θ):** Tight bound. Represents the average or exact growth rate when upper and lower bounds are the same.
- **Time vs. Space:** Time complexity measures operations; Space complexity measures *extra* memory used by the algorithm.
- **Hierarchy of Growth:** $O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(2^N) < O(N!)$.

Common Traps & Confusions

- **Constants Don't Matter:** $O(100N)$ is still $O(N)$. $O(N^2 + 1000N)$ is still $O(N^2)$. We only care about the fastest-growing term.
- **Log Base:** In CS, `log N` usually refers to base 2. Since `log_a(N) = log_b(N) / log_b(a)`, the base is just a constant multiplier and is ignored in Big-O.
- **Amortized Analysis:** An "average" over a series of operations (e.g., dynamic array resizing is $O(N)$ occasionally but $O(1)$ on average).

MCQs

Q1. (Conceptual) What is the primary purpose of **Asymptotic Analysis** in computer science?

A) To calculate the exact execution time of a program in milliseconds.

B) To describe how the resource requirements (time/space) of an algorithm grow as the input size increases towards infinity.

C) To determine the physical RAM requirements of a software package.

D) To count the number of variables in a function.

> Hardware and compilers change, so measuring time in seconds is useless. Asymptotic analysis provides a mathematical way to compare algorithms regardless of the machine they run on.

Q2. (Calculation) What is the Big-O complexity of an algorithm that performs `3N² + 50N + 1000` operations?

A) O(N)

B) O(50N)

C) O(N²)

D) O(1000)

> In asymptotic analysis, we drop all lower-order terms (50N, 1000) and all constant coefficients (3). The dominant term is `N²`.

Q3. (Scenario-Based) A developer is choosing between two search algorithms. Algorithm A is `O(N)` and Algorithm B is `O(log N)`. For an input of 1,000,000 items, which is significantly more efficient?

A) Algorithm A

B) Algorithm B

C) Both are equally efficient for large data.

D) Not enough information.

> For N=1,000,000, Algorithm A takes ~1,000,000 steps. Algorithm B (log base 2) takes only ~20 steps. Logarithmic growth is vastly superior to linear growth for large datasets.

Q4. (Code-Tracing) What is the time complexity of the following loop?

```
for (int i = 1; i < n; i = i * 2) {
    // constant time operation
}
```

A) O(n)

B) O(n²)

C) O(log n)

D) O(1)

> The loop variable `i` doubles in every iteration (1, 2, 4, 8, 16...). The number of times you can double a number before hitting `n` is `log2(n)`.

Q5. (Conceptual) If an algorithm is said to have **Amortized O(1)** complexity for an operation, what does this mean?

A) Every single call to the operation takes constant time.

B) While a single operation might occasionally be expensive, the average time per operation over a long sequence of calls is constant.

C) The algorithm uses O(1) extra space.

D) The algorithm only works for small values of N.

> A classic example is adding an item to an `ArrayList`. Usually it is O(1). Occasionally, the array is full and must be copied (O(N)). However, over 1000 additions, the total time is very low, averaging to O(1).

Q6. (Code-Tracing) Analyze the complexity of this nested loop:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < 100; j++) {
        print(i, j);
    }
}
```

A) $O(n^2)$

B) O(100n)

C) O(n)

D) O(1)

> The outer loop runs `n` times. The inner loop runs exactly `100` times, which is a constant and does not grow with `n`. Thus, the total complexity is `n * 100`, which simplifies to `O(n)`.

Q7. (Conceptual) Which notation is used to describe the **Best Case** scenario of an algorithm's growth?

A) Big-O (O)

B) Big-Theta (Θ)

C) Big-Omega (Ω)

D) Little-o (o)

> Big-Omega (Ω) provides the lower bound. For example, the best case for Bubble Sort (on a sorted array) is `Ω(n)`, even though its worst case is `O(n²)`.

Q8. (Scenario-Based) An algorithm uses `O(n)` extra space to process an array of size `n`. If `n` increases from 1,000 to 1,000,000, what happens to the **extra memory** required?

A) It stays the same.

B) It increases by 20 times.

C) It increases by 1,000 times (linearly).

D) It increases by 1,000,000 times.

> Space complexity works exactly like time complexity. `O(n)` space means the memory required grows in direct proportion to the input size.

Q9. (Calculation) What is the complexity of an algorithm that first sorts an array using Merge Sort (O(n log n)) and then performs a linear search (O(n)) on it?

A) $O(n^2 \log n)$

B) $O(n^2)$

C) O(n log n)

D) O(n)

> Total complexity = `O(n log n + n)`. In Big-O, we only keep the dominant term. Since `n log n` grows faster than `n`, the result is `O(n log n)`.

Q10. (Debugging) A developer believes their algorithm is `O(1)` because it doesn't have any loops. However, inside the function, they call `list.contains(item)` on a list of size `n`. What is the actual complexity?

A) O(1)

B) O(n)

C) O(log n)

D) $O(n^2)$

> The complexity of a function includes the complexity of all the helper functions it calls. Since `list.contains()` is a linear search `O(n)`, the overall function becomes `O(n)`.

Q11. (Conceptual) In Big-O analysis, which of the following functions grows the **slowest**?

A) O(n)

B) O(log n)

C) O(n log n)

D) O(2^n)

> Logarithmic growth is very slow. Doubling `n` only adds 1 unit of work to `log n`, whereas it adds n units of work to a linear function.

Q12. (Code-Tracing) What is the time complexity of a recursive function that calls itself twice with `(n-1)` each time? (e.g., `f(n) { f(n-1); f(n-1); }`)

A) O(n)

B) O(n^2)

C) O(log n)

D) O(2^n)

> Each call spawns 2 more calls. This creates a binary tree of calls with depth `n`. The total number of nodes in such a tree is `2^n - 1`, resulting in Exponential complexity.

Q13. (Conceptual) When an algorithm is `Θ(n log n)`, it means:

A) The worst case is n log n.

B) The best case is n log n.

C) Both the upper and lower bounds are n log n (it is a tight bound).

D) The algorithm is inefficient.

> Big-Theta represents a tight bound. It means the algorithm's performance is always proportional to `n log n`, regardless of the input distribution.

Q14. (Calculation) An algorithm has a complexity of $O(n^2)$. If it takes 0.1 seconds to process 1,000 items, how long will it take to process 10,000 items?

A) 1 second

B) 10 seconds

C) 100 seconds

D) 0.2 seconds

> The input size n increased by 10x. Since the complexity is n^2, the time increases by $10^2 = 100$x.
> `0.1 seconds * 100 = 10 seconds`.

Q15. (Scenario-Based) You have an algorithm that uses a significant amount of extra RAM (O(n)) to achieve a very fast runtime (O(1)). This is a classic example of:

A) Recursive Optimization

B) Space-Time Tradeoff

C) Asymptotic Decay

D) Memory Fragmentation

> Engineers often trade memory (space) to gain speed (time). Hashing is the most common example of this tradeoff.

Deep Dive: Advanced Complexity Theory

1. The Formal Definitions

For graduates, understanding the mathematical definitions is key:

- **Big-O (Upper Bound):** $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n0 > 0$ such that $0 \leq f(n) \leq c*g(n)$ for all $n \geq n0$.
- **Big-Omega (Lower Bound):** $f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n0 > 0$ such that $0 \leq c*g(n) \leq f(n)$ for all $n \geq n0$.
- **Big-Theta (Tight Bound):** $f(n) = \Theta(g(n))$ if it is both $O(g(n))$ and $\Omega(g(n))$.

2. Recursive Complexity and the Master Theorem

To find the complexity of divide-and-conquer algorithms, we use the **Master Theorem** for recurrences of the form $T(n) = aT(n/b) + f(n)$:

- **Binary Search:** $T(n) = T(n/2) + O(1) \rightarrow O(\log n)$
- **Merge Sort:** $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$

3. Space Complexity - Stack vs. Heap

In complexity analysis, don't forget the **Call Stack**. A recursive function with no extra arrays but a recursion depth of n has a **Space Complexity of O(n)** because n stack frames are stored in memory simultaneously.

4. Practical Complexity Chart

N	log N	N	N log N	N^2	2^N
10	3	10	30	100	1,024
100	7	100	700	10,000	1.26×10^{30}
1,000	10	1,000	10,000	1,000,000	Impossible

Stable Web Resources

- Wikipedia: Big O Notation
- Big-O Cheat Sheet (Visualizer)
- GeeksforGeeks: Asymptotic Analysis

Quick Revision Cheat Sheet

- **O(1):** Constant. Access array by index, push to stack.
- **O(log n):** Logarithmic. Binary search.
- **O(n):** Linear. Iterating through a list once.
- **O(n log n):** Log-linear. Efficient sorting (Merge, Quick).
- **O(n^2):** Quadratic. Nested loops, Bubble sort.
- **O(2^n):** Exponential. Recursive Fibonacci, power set.
- **Space-Time:** Trade RAM to save CPU cycles.
- **Drop Constants:** O(5N + 20) → O(N).

END OF SUBTOPIC 01 — MOVING TO SUBTOPIC 02

Topic 02: Linear Data Structures (25 MCQs)

Quick Notes

- **Linear Structure:** Elements are arranged in a sequential order where each element (except first/last) has a unique predecessor and successor.
- **Arrays:** Contiguous memory, O(1) access, O(N) insertion/deletion.
- **Linked Lists:** Non-contiguous, O(N) access, O(1) insertion (if position is known). **Singly:** One-way; **Doubly:** Two-way.
- **Stack (LIFO):** Last-In-First-Out. Operations: Push, Pop, Peek.
- **Queue (FIFO):** First-In-First-Out. Operations: Enqueue, Dequeue.
- **Strings:** Linear arrays of characters. Often immutable (Java/Python) or mutable (C++).

Common Traps & Confusions

- **Array vs List:** Arrays are fixed-size; Linked Lists are dynamic. Arrays have better **Cache Locality**.
- **Stack/Queue Implementation:** Using an array for a Queue requires **Circular** logic to avoid O(N) dequeues.
- **String Comparison:** In many languages, == compares memory addresses (Reference), while `.equals()` compares text (Value).

MCQs

Q1. (Conceptual) Why is accessing an element by index in a standard Array an O(1) operation?

A) The computer performs a linear search.

B) The address is calculated using a constant-time mathematical formula (Base + Index * Size).

C) Arrays use a binary search internally.

D) Arrays are always stored in the CPU registers.

> Because arrays are contiguous, the CPU simply calculates the physical memory address directly and jumps to it.

Q2. (Code-Tracing) Trace the result of these Stack operations: `Push(10), Push(20), Pop(), Push(30), Peek()`.

A) 10

B) 20

C) 30

D) Error

> 1. [10] → 2. [10, 20] → 3. [10] (20 popped) → 4. [10, 30] → 5. Peek returns 30.

Q3. (Scenario-Based) Which data structure is most efficient for a "Undo" feature in a text editor?

A) Queue

B) Linked List

C) Stack

D) Array

> Undo requires reversing the *most recent* action, which matches the LIFO (Last-In-First-Out) property of a Stack.

Q4. (Conceptual) What is the time complexity of deleting the **first element** of a standard Array of size N?

A) O(1)

B) O(N)

C) O(log N)

D) $O(N^2)$

> When the first element is removed, every remaining element (N-1) must be shifted one position to the left.

Q5. (Calculation) In a 64-bit system, a Singly Linked List node stores an `int` (4 bytes) and a `next` pointer (8 bytes). What is the total memory used by 100 nodes?

A) 400 bytes
B) 800 bytes
C) 1200 bytes
D) 1000 bytes

> Each node = 4 (data) + 8 (pointer) = 12 bytes. 100 nodes * 12 bytes = 1200 bytes.

Q6. (Conceptual) Which structure uses the **FIFO** (First-In-First-Out) principle?

A) Stack
B) Queue
C) Linked List
D) String

> Queues work like a real-world line; the first one to join is the first one to be served.

Q7. (Scenario-Based) An application needs to store 1,000,000 records. It performs **millions of middle-insertions** but zero random-access lookups. Which is best?

A) Array
B) Linked List
C) Stack
D) String

> Linked Lists handle insertions in O(1) by just updating pointers, whereas arrays would require massive O(N) shifts.

Q8. (Code-Tracing) In a Circular Queue of size 5, if `front = 4` and `rear = 4` (full), and one item is dequeued, what is the new `front`?

A) 5
B) 0
C) 3
D) -1

> Front moves using `(front + 1) % size`.
> `(4 + 1) % 5 = 0`.

Q9. (Conceptual) Why are Strings often **Immutable** in modern languages like Java?

A) To make them slower.

B) To allow String Pooling, improve security, and ensure thread-safety.

C) Because memory cannot store letters.

D) To prevent users from typing.

> Immutability allows multiple variables to point to one string in the "Pool" without worrying that one will change it for the others.

Q10. (Comparison) What is the primary advantage of a **Doubly Linked List** over a Singly Linked List?

A) Uses less memory.

B) Allows traversal in both directions and O(1) deletion of a node if given its pointer.

C) It is faster for searching.

D) It doesn't need a head pointer.

> Because every node has a `prev` pointer, you can delete a node without traversing from the head to find its predecessor.

Q11. (Debugging) A programmer implements a Queue using an array. After many enqueues and dequeues, the "Queue Full" error triggers even though the array is mostly empty. Why?

A) Memory leak.

B) The rear pointer hit the end of the array; a Circular Queue was needed.

C) The array size is too small.

D) Integers are too large.

> In a non-circular array queue, the rear pointer only moves forward. Dequeued space at the front is wasted.

Q12. (Scenario-Based) A function needs to check if a sequence of brackets `{[()]}` is **Balanced**. Which structure should it use?

A) Stack

B) Queue

C) Linked List

D) Array

> Push opening brackets. When a closing bracket appears, Pop and check for a match. The LIFO nature ensures the innermost brackets are matched first.

Q13. (Calculation) What is the 1D index of `M[2][3]` in a Row-Major array of size `[5][10]`?

A) 5

B) 13

C) 23

D) 32

> Formula: `Row * TotalCols + Col`.
> `2 * 10 + 3 = 23`.

Q14. (Conceptual) What happens in a **Null Pointer Dereference**?

A) The program runs faster.

B) The program attempts to read from address 0x0 and crashes.

C) The variable is set to 1.

D) A new object is created.

> Accessing data through a pointer that points to nothing is a critical runtime error.

Q15. (Scenario-Based) You need to reverse a string. Which approach is most space-efficient?

A) Copy to a new string in reverse order.

B) Use two pointers (start and end) and swap characters in-place.

C) Push all characters to a Stack and Pop them.

D) Use recursion.

> Swapping in-place uses O(1) extra space, whereas a Stack or a new String uses O(N).

Q16. (Conceptual) In a Linked List, what does `head.next = head.next.next` effectively do?

A) Swaps the first two nodes.

B) Deletes the second node by bypassing it.

C) Deletes the entire list.

D) Adds a new node at the start.

> It changes the link of the first node to point to the third node, leaving the second node unreachable.

Q17. (Code-Tracing) A Queue is implemented using two Stacks (S1 for Enqueue, S2 for Dequeue). If S2 is empty, what must happen before a Dequeue?

A) Throw an underflow error.

B) Pop everything from S1 and Push it into S2.

C) Reverse S1 in-place.

D) Delete S1.

> Transferring from S1 to S2 reverses the LIFO order into FIFO order.

Q18. (Comparison) Why is a **Deque** more versatile than a standard Queue?

A) It uses less RAM.

B) It allows insertion and deletion from both the front and the rear.

C) It is always sorted.

D) It can only store strings.

> A Deque (Double-Ended Queue) can act as both a Stack and a Queue.

Q19. (Conceptual) What is a **Sentinel Node**?

A) A node that stores the size of the list.

B) A "dummy" node at the start or end used to simplify edge-case logic (like null checks).

C) A node that monitors the CPU.

D) A backup of the head node.

> Sentinels ensure the list is never "empty," making code cleaner by removing `if (head == null)` checks.

Q20. (Scenario-Based) An OS needs to manage a list of processes waiting for the CPU in order of arrival. Which structure is standard?

A) Stack

B) Queue

C) Array

D) Doubly Linked List

> Standard CPU scheduling (First-Come-First-Served) is a FIFO process.

Q21. (Code-Tracing) What is the **Top** of the stack after: `Push(A), Push(B), Pop(), Push(C)`?

A) A

B) B

C) C

D) Empty

> Sequence: [A] → [A, B] → [A] → [A, C]. Top is C.

Q22. (Calculation) How many pointers are updated when inserting a node into the middle of a **Singly** Linked List?

A) 1

B) 2

C) 4

D) 0

> 1. NewNode.next = Prev.next.
> 2. Prev.next = NewNode.

Q23. (Conceptual) What is **Cache Locality** and why do Arrays have it?

A) It refers to the speed of the hard drive.

B) It is the tendency of a CPU to access nearby memory locations; Arrays have it because their elements are contiguous.

C) It means the array is stored on a server.

D) It is a feature of Linked Lists.

> CPU reads memory in "lines." When you read `arr[0]`, `arr[1]` to `arr[15]` are also pulled into the cache, making iteration extremely fast.

Q24. (Scenario-Based) You need to store a very long string that you will frequently modify (insert/delete in the middle). Which is most efficient?

A) String Object

B) StringBuilder / char array

C) List of Integers

D) Stack

> Mutable structures allow changes without re-copying the entire string.

Q25. (Conceptual) What is the difference between a **Linear** and **Circular** Linked List?

A) Circular is faster for search.

B) In Circular, the last node points to the first; in Linear, it points to NULL.

C) Linear uses more memory.

D) Circular is only for numbers.

> Circular lists form a loop, which is useful for things like round-robin scheduling.

Deep Dive: Memory & Efficiency

1. The Pointer Overhead

In a Singly Linked List, 66% of the memory used is often just for pointers (on a 64-bit system). This makes them less memory-efficient than Arrays for small data types like `char` or `int`.

2. Amortized Analysis of Dynamic Arrays

Dynamic arrays double their size when full. While the "Resize" operation is O(N), it happens so rarely that the average cost per insertion remains O(1).

Stable Web Resources

- Wikipedia: Binary Search
- GeeksforGeeks: Searching Algorithms

Quick Revision Cheat Sheet

- **Array:** Fast access (O(1)), slow insertion (O(N)), cache-friendly.
- **Linked List:** Slow access (O(N)), fast insertion (O(1)).
- **Stack:** LIFO. Use for recursion/undo.
- **Queue:** FIFO. Use for scheduling.
- **Doubly List:** Two-way navigation. **Circular List:** No NULL at the end.

END OF SUBTOPIC 02 — MOVING TO SUBTOPIC 03

Topic 03: Non-Linear Data Structures (18 MCQs)

Quick Notes

- **Definition:** Data structures where elements are not arranged in a sequential or linear fashion. Each element can be attached to multiple other elements, representing hierarchical or networked relationships.
- **Trees (Hierarchical):** A collection of nodes starting from a single **Root**. Each node has zero or more child nodes. Trees are inherently recursive.

- **Graphs (Networked):** A set of **Vertices** (nodes) connected by **Edges**. Unlike trees, graphs can have cycles, multiple paths between nodes, and no specific "top" or "bottom."
- **Key Properties:** 1) **Degree:** In a tree, the number of children. In a graph, the number of edges connected to a vertex. 2) **Height:** The length of the longest path from a node to a leaf. 3) **Depth:** The length of the path from the root to a specific node.
- **Non-Linear vs. Linear:** Linear structures (Arrays/Lists) are 1-to-1. Non-linear structures are 1-to-many (Trees) or many-to-many (Graphs).

Common Traps & Confusions

- **Tree vs. Graph:** A tree is technically a "connected acyclic graph." If a graph is disconnected or has a cycle, it cannot be a tree.
- **Root and Direction:** Every tree must have exactly one root. In a graph, there is no root unless you specifically designate a "source" node for an algorithm.
- **Implementation:** While linear structures often use contiguous memory (Arrays), non-linear structures almost always rely on dynamic memory allocation and pointers.

MCQs

Q1. Which characteristic fundamentally distinguishes a Non-Linear data structure from a Linear one?

A) Non-linear structures can only store integers.

B) Linear structures are always stored on the heap.

C) In non-linear structures, an element can be associated with more than one predecessor or successor.

D) Non-linear structures do not use pointers.

> Linear structures like Arrays or Linked Lists have a unique next and previous element. Non-linear structures like Trees and Graphs allow branching connections.

Q2. In a mathematical Tree with N nodes, what is the exact number of edges required to keep it connected without cycles?

A) N

B) N - 1

C) N + 1

D) Log N

> Every node in a tree, except the root, has exactly one incoming edge from its parent. Therefore, N nodes require N-1 edges.

Q3. You are designing the folder structure for a new Operating System. Which non-linear data structure is the most natural representation for this hierarchy?

A) Graph

B) Stack

C) Tree

D) Queue

> Folders and sub-folders follow a parent-child relationship with a single root (the drive), which is the definition of a Tree.

Q4. Which structure would be best to model a "Social Network" where users can have many friends and those friends can also be friends with each other?

A) Binary Search Tree

B) Linked List

C) Graph

D) Min-Heap

> Social networks have "many-to-many" relationships and cycles (A is friends with B, B with C, and C with A), which can only be modeled by a Graph.

Q5. What is the **Degree** of a Leaf node in a tree?

A) 1

B) 0

C) 2

D) Infinite

> The degree of a node in a tree is the number of children it has. Since a leaf node has no children, its degree is 0.

Q6. In graph theory, an edge that connects a vertex to itself is known as a:

A) Bridge

B) Path

C) Self-loop

D) Cycle

> A self-loop is an edge (v, v). It is often used in state machines to represent staying in the current state.

Q7. Which graph representation is most memory-efficient for a **Sparse Graph** (a graph with very few edges compared to vertices)?

A) Adjacency Matrix

B) Adjacency List

C) 2D Array

D) All are equal

> An Adjacency Matrix uses V^2 space regardless of edges. An Adjacency List only uses space proportional to the number of vertices and edges (V + E).

Q8. A "Directed Acyclic Graph" (DAG) is a graph that:

A) Has no edges.

B) Is always a tree.

C) Has directed edges and contains no cycles.

D) Has edges pointing in both directions.

> DAGs are crucial in computing for representing dependencies, such as the order of tasks in a build system or courses with prerequisites.

Q9. In a tree, the **Height** of the root node is equivalent to:

A) The degree of the root.

B) The maximum depth of the tree.

C) The number of leaf nodes.

D) Always 0.

> The height of a node is the length of the longest path to a leaf. For the root, this spans the entire tree, matching the tree's overall depth.

Q10. A graph where every single vertex is connected to every other vertex is called a:

A) Bipartite Graph

B) Simple Graph

C) Complete Graph

D) Connected Forest

> In a complete graph with V vertices, there are V(V-1)/2 edges, ensuring every pair of nodes has a direct link.

Q11. Which statement about **Non-Linear** traversal is true?

A) It can only be done using a for-loop.

B) It typically requires a mechanism to track "visited" nodes to avoid infinite loops in graphs.

C) It is always O(1) time complexity.

D) It only works on trees, not graphs.

> Because graphs can have cycles, an algorithm must remember where it has already been (using a set or boolean array), or it might loop forever.

Q12. A **Forest** is mathematically defined as a:

A) Single very large tree.

B) Graph with a cycle.

C) Collection of disjoint (disconnected) trees.

D) Graph where every node has the same degree.

> A forest is simply an acyclic graph. Each connected component within that graph is a tree.

Q13. In a tree structure, what are **Siblings**?

A) Nodes at the same level.

B) Nodes that share the same immediate parent.

C) Nodes connected in a cycle.

D) The root and its leaves.

> Sibling nodes are nodes that originate from the same parent node. Nodes at the same level but with different parents are not siblings.

Q14. If you want to find the shortest path in an **Unweighted** graph, which traversal logic is most efficient?

A) Depth-First Search (DFS)

B) Breadth-First Search (BFS)

C) In-order Traversal

D) Pre-order Traversal

> BFS explores nodes in layers (distance 1, then 2, etc.). The first time it reaches a target node, it is guaranteed to be the shortest path in an unweighted graph.

Q15. What is a **Path** in a graph?

A) A sequence of vertices where each adjacent pair is connected by an edge.

B) The total number of edges in the graph.

C) A cycle that visits every node once.

D) The link between a parent and child in a tree.

> A path describes a route from one vertex to another following the edges of the graph.

Q16. Why is an **Adjacency Matrix** often considered inefficient for large-scale real-world graphs like the internet?

A) It is too difficult to program.

B) It requires $O(V^2)$ space, meaning most of the memory is wasted on zeros for non-existent connections.

C) It cannot store directed edges.

D) It makes searching for neighbors O(1).

> Real-world graphs are usually sparse. A matrix for 1 million nodes would require 1 trillion cells, most of which would be empty.

Q17. A tree where every internal node has **exactly** two children is called a:

A) Full Binary Tree

B) Complete Binary Tree

C) Perfect Binary Tree

D) Balanced Tree

> In a Full Binary Tree, every node has either 0 or 2 children. It is "full" because no node has only 1 child.

Q18. What is the "In-degree" of a vertex in a Directed Graph?

A) The number of nodes it can reach.

B) The number of edges pointing toward that vertex.

C) The value stored in the node.

D) The distance from the source.

> In-degree counts incoming edges. In a task dependency graph, a node with an in-degree of 0 is a task that can be started immediately.

Deep Dive: The Physical Reality of Non-Linearity

1. The "Contiguity" Problem

In linear structures like Arrays, the CPU loves the data because it is side-by-side (Contiguous). In Non-Linear structures, nodes are usually scattered across the **Heap**. Each node is a small island in RAM. To move from a Parent to a Child, the CPU must "dereference" a pointer, which means jumping to a completely different memory address. This can lead to **Cache Misses**, making Trees and Graphs significantly slower than Arrays for simple data processing.

2. Mathematical Constraints for Graduates

- **Handshaking Lemma:** In any graph, the sum of all vertex degrees is exactly twice the number of edges. This is because every edge is counted twice (once for each endpoint).
- **Tree Property:** For any connected graph, if you have V vertices and V-1 edges, it is *guaranteed* to be a tree.

- **Sparse vs. Dense:** A graph is "Sparse" if E is close to V. It is "Dense" if E is close to V squared. This distinction determines whether you should use an Adjacency List (Sparse) or Matrix (Dense).

3. The Recursive Nature

Non-linear structures are best processed using **Recursion**. A Tree is just a Root node connected to several smaller Trees (sub-trees). This is why algorithms for Trees are almost always written in 5-10 lines of recursive code, whereas iterative versions require complex manual Stacks.

Stable Web Resources

- Wikipedia: Non-linear Data Structures
- GeeksforGeeks: Comprehensive Graph Guide
- VisuAlgo: Visualizing Data Structures and Algorithms

Quick Revision Cheat Sheet

- **Non-Linear:** Trees and Graphs. association is 1-to-many or many-to-many.
- **Tree:** Connected, no cycles, N-1 edges. Hierarchical logic.
- **Graph:** Vertices and Edges. Networking logic. Can be directed/weighted.
- **Degree:** Children count (Tree) or Neighbor count (Graph).
- **Leaf:** A node with no children (Degree 0).
- **Traversal:** Needs a Queue (BFS) or Stack (DFS).
- **Adjacency Matrix:** Fast lookup, high memory.
- **Adjacency List:** Slow lookup, low memory.

END OF SUBTOPIC 03 — MOVING TO SUBTOPIC 04

Topic 04: Searching Algorithms (15 MCQs)

Quick Notes

- **Linear Search:** The simplest method; checks every element sequentially. Works on unsorted data. Time Complexity: O(N).
- **Binary Search:** A divide-and-conquer algorithm for **sorted** data. It halves the search space in every step. Time Complexity: O(log N).
- **Jump Search:** Jumps ahead by fixed blocks (e.g., of size √N) and then performs a linear search within the block. Requires sorted data. Time Complexity: O(√N).
- **Interpolation Search:** An improved version of Binary Search for **uniformly distributed** sorted data. It "guesses" the position of the target using a formula. Time Complexity: O(log log N).
- **Exponential Search:** Useful for searching in infinite or unbounded sorted lists. It finds a range where the target may exist and then performs Binary Search.
- **Ternary Search:** Divides the sorted array into three parts instead of two. While mathematically O(log3 N), it is often slower than Binary Search in practice due to more comparisons.

Common Traps & Confusions

- **The "Sorted" Constraint:** Binary, Jump, and Interpolation searches **all require sorted data**. Using them on unsorted arrays will result in incorrect logical outcomes.
- **Integer Overflow:** In many languages, calculating the midpoint as `(low + high) / 2` can cause a crash if the sum exceeds the integer limit. The professional way is `low + (high - low) / 2`.
- **Interpolation Failure:** Interpolation search is extremely fast for sequential data (1, 2, 3...) but can degrade to O(N) if the data is heavily skewed or not uniform.

MCQs

Q1. What is the absolute prerequisite for an array before a **Binary Search** can be successfully executed?

A) The array must contain only positive integers.

B) The array size must be a power of 2.

C) The array must be sorted in ascending or descending order.

D) The array must not contain duplicate values.

> Binary Search relies on the property that if the middle element is smaller than the target, the target must be in the right half. This logic is only true if the data is ordered.

Q2. In the worst-case scenario, how many comparisons are required to find an element in a sorted array of **1,024** elements using Binary Search?

A) 512

B) 1,024

C) 11

D) 20

> The maximum steps for Binary Search is ceil(log2 N) + 1. log2(1024) is 10. Adding the final check makes it roughly 11 comparisons.

Q3. You are given an **unsorted** dataset of 1,000,000 records. You need to search for a specific record **only once**. Which strategy is most time-efficient?

A) Perform a Linear Search O(N).

B) Sort the data O(N log N) and then use Binary Search O(log N).

C) Use Interpolation Search.

D) Convert the data to a Binary Search Tree.

> If you only search once, O(N) is faster than O(N log N). Sorting is only worth the time if you plan to perform multiple searches later.

Q4. Which search algorithm is specifically optimized for **Uniformly Distributed** sorted data, such as a numerical sequence or a phone book?

A) Jump Search

B) Linear Search

C) Interpolation Search

D) Ternary Search

> Interpolation search uses a slope-intercept formula to estimate where the target might be (similar to how humans open a dictionary near the end for the letter 'Z').

Q5. Trace a Binary Search for **target = 7** in the array `[1, 3, 5, 7, 9, 11, 13]`. What is the value of the `mid` index in the first iteration (using 0-based indexing)?

A) 0

B) 2

C) 3

D) 4

> Indices are 0 to 6. Mid = (0 + 6) / 2 = 3. `arr[3]` is 7. The target is found in the very first step.

Q6. What is the time complexity of the **Jump Search** algorithm when using the optimal block size of √N?

A) O(log N)

B) O(N)

C) O(√N)

D) O(1)

> Jump search jumps √N times to find the block, then does a linear search of at most √N elements. √N + √N results in O(√N).

Q7. Which algorithm is most suitable for searching an element in a sorted list of **unknown or infinite length**?

A) Binary Search

B) Exponential Search

C) Interpolation Search

D) Jump Search

> Exponential Search finds the upper bound of the search range by doubling the index (1, 2, 4, 8, 16...) until it exceeds the target, then performs Binary Search in that limited range.

Q8. While **Ternary Search** has a smaller logarithmic base (log3) than Binary Search, why is it generally **slower** in practice?

A) It uses more RAM.

B) It requires more comparisons (two instead of one) per iteration to decide which third of the array to keep.

C) It only works on floating point numbers.

D) It cannot be implemented recursively.

> Big-O ignores constants. Even though the number of iterations is fewer, the "work per iteration" is higher in Ternary Search, making it less efficient on modern CPUs.

Q9. In professional systems, why is `mid = low + (high - low) / 2` used instead of `mid = (low + high) / 2`?

A) To make the code run faster.

B) To prevent integer overflow when searching in extremely large arrays.

C) To ensure the result is always a float.

D) To handle negative numbers in the array.

> If `low` and `high` are very large (e.g., 1.5 billion each), their sum will exceed the 2.1 billion limit of a 32-bit integer, resulting in a wrap-around error.

Q10. What is the **Space Complexity** of an iterative implementation of Binary Search?

A) O(1)

B) O(log N)

C) O(N)

D) O(N log N)

> The iterative version only uses three integer variables (low, high, mid) regardless of the input size, making its extra space usage constant.

Q11. When is **Linear Search** more appropriate than Binary Search?

A) When the array is very large.

B) When the data is unsorted and we only need to search once.

C) When memory usage must be minimized.

D) When the computer is 64-bit.

> Linear search has zero overhead for sorting. If the data is not already sorted, sorting it just to perform one search is mathematically wasteful.

Q12. Searching for a value in a **Binary Search Tree (BST)** follows a logic most similar to which searching algorithm?

A) Linear Search
B) Interpolation Search
C) Binary Search
D) Jump Search

> Just like Binary Search, at every node in a BST, you discard half of the remaining nodes based on whether the target is smaller or larger than the current node.

Q13. If a dataset is sorted but the values are `[1, 2, 3, 4, 1000000]`, which algorithm will perform **worst** compared to its average case?

A) Binary Search
B) Linear Search
C) Interpolation Search
D) Jump Search

> Interpolation search relies on a uniform distribution. The sudden jump to 1,000,000 (an outlier) will confuse the "guess" formula, causing it to perform closer to O(N).

Q14. For an array of size 10,000, what is the most efficient block size for a **Jump Search**?

A) 10
B) 100
C) 1,000
D) 5,000

> The optimal step size is √N. Since the square root of 10,000 is 100, that is the most efficient jump interval.

Q15. In terms of **Time-Space tradeoff**, which of the following is true about searching?

A) Linear search uses the most memory.

B) We can achieve O(1) search time by using a Hash Table, but at the cost of significantly higher memory usage.

C) Binary search is O(1) in space and time.

D) Sorting data reduces memory usage.

> Hashing is the ultimate search optimization, offering constant time lookups by "wasting" memory to create a direct mapping from keys to indices.

Deep Dive: The Information Theory of Search

1. Reducing Entropy

Searching is essentially the process of reducing uncertainty.
In **Linear Search**, you gain 1 bit of information per comparison (e.g., "The target is not at this index").
In **Binary Search**, you gain maximum information—exactly half the remaining search space—in every single step. This is why Binary Search on the entire human population (8 billion people) would only take 33 comparisons.

2. The Physics of Midpoint Selection

Selecting the midpoint as `low + (high - low) / 2` is a standard safety protocol in Google, Microsoft, and Linux kernels. It handles the **Integer Overflow** bug that went unnoticed in Java's `Arrays.binarySearch()` for nearly 10 years before being fixed.

3. Search Selection Matrix

Data State	Data Distribution	Best Choice
Unsorted	Any	Linear Search
Sorted	Non-uniform (Random)	Binary Search
Sorted	Uniformly spread	Interpolation Search
Unbounded/Stream	Sorted	Exponential Search

Stable Web Resources

- Wikipedia: Binary Search Algorithm
- GeeksforGeeks: Searching Algorithms Guide
- Brilliant.org: Binary Search Logic

Quick Revision Cheat Sheet

- **Linear Search:** O(N). The only option for unsorted data.
- **Binary Search:** O(log N). Requires sorted data. Divide and conquer.
- **Jump Search:** O(√N). Jump by √N, then linear search.
- **Interpolation Search:** O(log log N). "Guess" the position based on value.
- **Recursive Binary Search:** Uses O(log N) stack space. Iterative uses O(1).
- **Search Failure:** If `low > high`, the element is not in the list.

END OF SUBTOPIC 04 — MOVING TO SUBTOPIC 05

Topic 05: Sorting Algorithms (20 MCQs)

Quick Notes

- **Stable vs. Unstable:** A **Stable** sort preserves the relative order of elements with equal values (e.g., Merge Sort). **Unstable** does not (e.g., Quick Sort).
- **In-Place Sort:** An algorithm that requires only a constant amount O(1) of extra memory space (e.g., Bubble Sort, Quick Sort).
- **Comparison-based Sorts:** Minimum mathematical complexity is O(N \log N). Examples: Merge Sort, Quick Sort, Heap Sort.
- **Non-Comparison Sorts:** Can achieve O(N) by making assumptions about the data. Examples: Counting Sort, Radix Sort.

- **Adaptive Sort:** An algorithm that runs faster if the input is already partially sorted (e.g., Insertion Sort).

Common Traps & Confusions

- **Quick Sort Worst Case:** Quick Sort is O(N \log N) on average, but if the pivot is poorly chosen (e.g., always the smallest/largest element), it degrades to O(N^2).
- **Merge Sort Memory:** Unlike Quick Sort, Merge Sort is NOT in-place. It requires an auxiliary array of size O(N).
- **Small Data Efficiency:** For very small arrays (N < 20), simple algorithms like **Insertion Sort** are actually faster than Merge or Quick sort due to lower overhead.

MCQs

Q1. (Conceptual) What does it mean for a sorting algorithm to be **"Stable"**?

A) It runs in the same amount of time for all inputs.

B) It preserves the original relative order of elements that have equal keys.

C) It uses a constant amount of extra memory.

D) It never crashes during execution.

> Stability is crucial when sorting data by multiple criteria (e.g., sorting by name, then sorting by department). A stable sort ensures the names remain alphabetical within each department.

Q2. (Calculation) What is the **Worst-Case** time complexity of **Quick Sort**?

A) O(N)

B) O(N log N)

C) $O(N^2)$

D) O(2^N)

> Worst case occurs when the pivot consistently results in highly unbalanced partitions (e.g., sorting a already sorted array using the first element as pivot). Average case is O(N \log N).

Q3. (Scenario-Based) You have a 100GB file of data to sort, but your computer only has 8GB of RAM. Which sorting strategy is designed for this situation?

A) Quick Sort

B) Heap Sort

C) External Merge Sort

D) Insertion Sort

> External sorting handles data that doesn't fit in RAM. It sorts small "chunks" in memory, saves them to disk, and then merges the sorted files.

Q4. (Conceptual) Which of the following algorithms is **Non-Comparison based** and can achieve O(N) time complexity?

A) Bubble Sort

B) Merge Sort

C) Counting Sort

D) Quick Sort

> Counting Sort doesn't compare elements. It counts the occurrences of each value. It is very fast but only works when the range of input values is known and not too large.

Q5. (Code-Tracing) After the **first pass** of **Bubble Sort** (ascending) on array `[5, 1, 4, 2, 8]`, what is the resulting state?

A) [1, 2, 4, 5, 8]

B) [1, 4, 2, 5, 8]

C) [5, 4, 2, 1, 8]

D) [1, 5, 4, 2, 8]

> Pass 1: (5,1)→1,5; (5,4)→4,5; (5,2)→2,5; (5,8)→no swap. The largest element (8) has "bubbled" to the end.

Q6. (Comparison) Why is **Merge Sort** preferred for sorting **Linked Lists** over **Quick Sort**?

A) It uses less memory.

B) It accesses data sequentially, which is efficient for Linked Lists where random access is slow.

C) It is an in-place algorithm.

D) It is always faster than Quick Sort.

> Merge Sort's pattern of dividing and merging works perfectly with the sequential nature of pointers in a Linked List. Quick Sort requires random access to pick pivots and swap, which is O(N) for lists.

Q7. (Conceptual) Which sorting algorithm is **Adaptive** (meaning it becomes O(N) for nearly-sorted data)?

A) Selection Sort

B) Heap Sort

C) Insertion Sort

D) Merge Sort

> Insertion sort only performs swaps if an element is out of place. If the data is already sorted, it simply traverses the array once, resulting in linear time.

Q8. (Debugging) A developer chooses **Selection Sort** because it has a low number of swaps. What is the major drawback of this algorithm?

A) It is unstable.

B) It is always $O(N^2)$, even if the data is already perfectly sorted.

C) It uses O(N) extra memory.

D) It can only sort integers.

> Selection sort must scan the remaining unsorted part to find the minimum in every iteration. It never "realizes" the array is already sorted, making it inefficient.

Q9. (Calculation) How many **comparisons** does **Insertion Sort** perform in the **Best Case** (already sorted array of size N)?

A) 0

B) N - 1

C) N log N

D) N^2

> For an array of size N, it checks each element once against its predecessor (starting from the second element) and finds no swaps are needed.

Q10. (Conceptual) Which sorting algorithm utilizes a **Max-Heap** data structure to maintain the unsorted part of the array?

A) Quick Sort

B) Merge Sort

C) Heap Sort

D) Radix Sort

> Heap Sort builds a Max-Heap (O(N)), then repeatedly extracts the root (largest element) and moves it to the end of the array (O(N \log N) total).

Q11. (Comparison) Which of the following O(N \log N) algorithms is **not in-place** (requires O(N) extra space)?

A) Quick Sort

B) Heap Sort

C) Merge Sort

D) Shell Sort

> Merge Sort requires a temporary "auxiliary" array to hold the elements while they are being merged back in order.

Q12. (Scenario-Based) You need an algorithm with a **guaranteed** O(N \log N) worst-case time complexity and **O(1)** extra space. Which is the best choice?

A) Merge Sort

B) Quick Sort

C) Heap Sort

D) Counting Sort

> Merge sort uses O(N) space. Quick sort has an O(N^2) worst case. Heap sort is the only one that satisfies both requirements.

Q13. (Code-Tracing) In **Quick Sort**, what is the role of the **Partition** step?

A) To sort the entire array.

B) To place the pivot in its correct sorted position and move smaller elements to its left and larger to its right.

C) To divide the array into two equal halves.

D) To find the median value.

> Partitioning is the "workhorse" of Quick Sort. It organizes the data around a pivot value.

Q14. (Conceptual) Radix Sort sorts numbers by processing individual digits. Which sorting algorithm is typically used as the "stable" subroutine for Radix Sort?

A) Selection Sort

B) Quick Sort

C) Counting Sort

D) Bubble Sort

> Radix sort processes digits from least significant to most significant. It requires a stable internal sort to maintain order from previous digit passes; Counting Sort is the standard choice.

Q15. (Scenario-Based) An array has only 3 distinct values (e.g., 0, 1, 2) repeated many times. Which algorithm (Dutch National Flag problem) is most efficient?

A) Standard Quick Sort

B) Three-way Partitioning (Variation of Quick Sort)

C) Merge Sort

D) Bubble Sort

> Three-way partitioning is an optimization that handles arrays with many duplicate keys in O(N) time.

Q16. (Calculation) What is the time complexity of **Heapify** (converting an array into a heap)?

A) O(log N)

B) O(N)

C) O(N log N)

D) O(1)

> While inserting N elements one by one into a heap is O(N \log N), the bottom-up "Heapify" approach is mathematically proven to be O(N).

Q17. (Conceptual) Which sorting algorithm is commonly used as the default sort in many languages (e.g., Python's `sort()`, Java's `Arrays.sort()` for objects)?

A) Quick Sort

B) Merge Sort

C) Timsort

D) Insertion Sort

> Timsort is a hybrid algorithm (derived from Merge Sort and Insertion Sort) designed to perform very well on real-world data which often contains pre-existing sorted segments (runs).

Q18. (Debugging) You implemented Quick Sort, and it works, but it takes a long time for large, already-sorted arrays. How can you fix this?

A) Switch to Bubble Sort.

B) Use a random element or "Median-of-Three" as the pivot instead of the first element.

C) Use recursion more often.

D) Remove all comments.

> Picking the first or last element as pivot is risky. Randomized or median pivots prevent the O(N^2) worst case for sorted or nearly-sorted data.

Q19. (Conceptual) Which sorting algorithm is **Online** (can sort data as it is received from a stream)?

A) Selection Sort

B) Heap Sort

C) Insertion Sort

D) Quick Sort

> Insertion sort can maintain a sorted list as new elements are added one by one, making it "online."

Q20. (Comparison) What is the **Recurrence Relation** for Merge Sort?

A) T(n) = T(n-1) + O(1)

B) T(n) = T(n/2) + O(1)

C) T(n) = 2T(n/2) + O(n)

D) T(n) = 2T(n/2) + O(1)

> Merge sort divides the problem into two halves (2T(n/2)) and then performs an O(n) merge operation. This results in O(n \log n) total complexity.

Deep Dive: Sorting Mechanics

1. Stability in the Real World

Stability isn't just theory. Imagine you have a list of Objects: `{Order: 1, Price: 10}`, `{Order: 2, Price: 10}`. If you sort by Price using an **Unstable** sort, Order 2 might end up before Order 1. For data integrity in databases, **Stable** sorts (like Merge Sort) are vital.

2. Average vs. Worst Case

Why use Quick Sort if it has a worst case of O(N^2)?
Answer: **Constants.** Quick Sort has a very small constant factor and excellent **Cache Locality**. It sorts in-place and is usually faster than Merge or Heap sort on physical hardware for random data.

3. Hybrid Algorithms

Modern production sorts are rarely "pure."

- **Timsort:** Uses Merge Sort, but switches to Insertion Sort for small chunks (runs).
- **Introsort:** Starts with Quick Sort, but switches to Heap Sort if the recursion depth exceeds a certain level (to avoid O(N^2)).

Stable Web Resources

- Wikipedia: Sorting Algorithms
- Toptal: Sorting Algorithms Animations
- VisuAlgo: Sorting Visualization

Quick Revision Cheat Sheet

Algorithm	Best Case	Average/Worst	Space	Stable?
Bubble/Insertion	O(N)	$O(N^2)$	O(1)	Yes
Selection	$O(N^2)$	$O(N^2)$	O(1)	No
Merge	O(N log N)	O(N log N)	O(N)	Yes
Quick	O(N log N)	$O(N^2)$	O(log N)	No
Heap	O(N log N)	O(N log N)	O(1)	No

END OF SUBTOPIC 05 — MOVING TO SUBTOPIC 06

Topic 06: Hashing (15 MCQs)

Quick Notes

- **Definition:** Mapping data of arbitrary size (Key) to fixed-size values (Hash) using a **Hash Function**.
- **Hash Table:** A data structure that uses hashing to provide O(1) average time for search, insertion, and deletion.
- **Collision:** When two different keys produce the same hash index.
- **Collision Resolution:** 1) **Chaining (Open Hashing):** Each bucket contains a Linked List of all items with the same hash. 2) **Open Addressing (Closed Hashing):** If a collision occurs, find another empty slot in the array (Linear Probing, Quadratic Probing, Double Hashing).
- **Load Factor (\alpha):** n/m (items/buckets). High load factor increases collisions and slows down the table.

Common Traps & Confusions

- **Linear Probing Clustering:** Linear probing causes "Primary Clustering" where long runs of occupied slots build up, slowing down search.
- **Delete in Open Addressing:** You cannot just "null" a slot; you must use a special "Deleted" marker (Dummy) so the search algorithm knows to keep looking for items further down the probe sequence.
- **Perfect Hashing:** A hash function that results in zero collisions. Only possible if the set of keys is static and known in advance.

MCQs

Q1. (Conceptual) What is the main goal of a **Hash Function** in a Hash Table?

A) To encrypt data so it cannot be read.

B) To distribute keys uniformly across the available buckets in the table.

C) To sort the keys in ascending order.

D) To compress the data to take up less space.

> A good hash function minimizes collisions by spreading keys evenly throughout the array.

Q2. (Calculation) You have a Hash Table of size 10. Using the hash function `h(k) = k mod 10`, what is the index for key **1257**?

A) 1

B) 2

C) 5

D) 7

> `1257 % 10 = 7`. The remainder of the division by the table size is the index.

Q3. (Conceptual) Which collision resolution technique uses a **Linked List** at each bucket to handle multiple keys?

A) Separate Chaining

B) Linear Probing

C) Quadratic Probing

D) Double Hashing

> In Chaining, each slot in the hash table points to the head of a linked list. All keys that hash to that index are stored in the list.

Q4. (Debugging) A developer notices that a `HashMap` search is becoming very slow (O(N)). What is the most likely cause?

A) The table size is too small.

B) Poor hash function choice causing many collisions (clustering into a single bucket).

C) The keys are integers instead of strings.

D) The computer has too much RAM.

> In the worst case (all keys hash to the same bucket), a hash table degrades into a Linked List, changing the search complexity from O(1) to O(N).

Q5. (Conceptual) In **Open Addressing**, what is **Linear Probing**?

A) Moving to a random empty slot.

B) Searching for the next available slot sequentially (index + 1, index + 2...).

C) Using a second hash function to find a slot.

D) Creating a new array.

> Linear probing is the simplest open addressing method. If index `i` is full, it checks `i+1`, then `i+2`, wrapping around at the end.

Q6. (Scenario-Based) A system needs to handle a **Load Factor of 2.0**. Which collision resolution strategy is the only one that can handle this?

A) Linear Probing

B) Double Hashing

C) Separate Chaining

D) Quadratic Probing

> Load factor \alpha = n/m. If n > m (more items than slots), Open Addressing fails because there are no empty slots. Chaining can handle an infinite load factor because the lists can grow indefinitely.

Q7. (Conceptual) What is **Primary Clustering** in Hash Tables?

A) When the first index is always full.

B) The buildup of long contiguous blocks of occupied slots in linear probing, which increases search time.

C) When the hash function only returns prime numbers.

D) When the table size is a power of two.

> Primary clustering is a major disadvantage of linear probing. Once a block forms, any key hashing into that block will make the block even longer.

Q8. (Comparison) What is the main advantage of **Double Hashing** over **Linear Probing**?

A) It is easier to implement.

B) It significantly reduces clustering by using a second hash function to determine the "step size" for probes.

C) it uses less memory.

D) It doesn't require a table size.

> Double hashing ensures that keys that hash to the same initial index have different probe sequences, avoiding both primary and secondary clustering.

Q9. (Conceptual) Why is it recommended to use a **Prime Number** for the size of a Hash Table?

A) Primes take up less memory.

B) It helps in distributing keys more uniformly, especially if the keys have patterns related to powers of 2.

C) Compilers can only divide by prime numbers.

D) It prevents users from hacking the data.

> Using a prime table size reduces the likelihood that patterns in the input keys (like all being even) will align with the table size and cause collisions.

Q10. (Scenario-Based) You are using Open Addressing. You delete an element at index 5. Why shouldn't you just set `table[5] = NULL`?

A) It causes a memory leak.

B) It may "break" the probe sequence, making other items that were placed after index 5 unreachable during a search.

C) NULL values are not allowed in arrays.

D) Setting values to NULL is slow.

> If a search for key K passed through index 5 during its initial probe, setting it to NULL will make the search algorithm stop there, incorrectly reporting that K is missing.

Q11. (Calculation) If a Hash Table has 100 slots and contains 70 items, what is its **Load Factor**?

A) 1.42

B) 0.7

C) 70

D) 30

> \alpha = \text{Items} / \text{Slots} = 70 / 100 = 0.7.

Q12. (Conceptual) What is **Rehashing**?

A) Running the hash function twice for security.

B) Creating a larger table and moving all existing items to it when the load factor exceeds a certain threshold.

C) Deleting the whole table.

D) Changing the data type of the keys.

> To maintain O(1) performance, hash tables must remain sparse. When they get too full (e.g., \alpha > 0.75), they double in size and re-calculate the index for every item.

Q13. (Comparison) Compare the Space Complexity of Chaining vs. Open Addressing.

A) Open Addressing uses more space.

B) Chaining uses extra space for pointers; Open Addressing uses no extra space outside the array but requires a larger initial array.

C) They are identical.

D) Chaining uses less RAM.

> Chaining requires extra memory for each `Node` (Data + Pointer). Open Addressing is more space-efficient per element but requires the array to be kept quite empty to stay fast.

Q14. (Conceptual) In **Quadratic Probing**, if the initial hash is `h`, the sequence of probes is:

A) h+1, h+2, h+3...

B) $h+1^2$, $h+2^2$, $h+3^2$...

C) h*1, h*2, h*3...

D) h+k1, h+k2...

> Quadratic probing reduces primary clustering by increasing the distance between probes non-linearly.

Q15. (Scenario-Based) Which of the following is a "Cryptographic" hash function requirement, but NOT necessarily required for a standard Hash Table?

A) Uniform distribution

B) Fast computation

C) Pre-image resistance (Impossible to reverse the hash to find the key)

D) Determinism (Same key always yields same hash)

> Standard hash tables only care about speed and distribution. Cryptographic hashes (like SHA-256) must also be one-way and secure against tampering.

Deep Dive: The Science of Hashing

1. The "Magic" of Constant Time O(1)

How does a Hash Table achieve O(1)? By turning a **Logical Search** (finding a name) into a **Physical Index** (jumping to array index 5). As long as the hash function is fast and the table is not full, the search time does not grow even if you have 10 billion items.

2. Probing Strategies Compared

- **Linear Probing:** Cache-friendly (data is side-by-side) but suffers from **Primary Clustering**.
- **Quadratic Probing:** Avoids primary clustering but can suffer from **Secondary Clustering** (keys that hash to the same spot follow the same path).
- **Double Hashing:** Best distribution, avoids both clusters, but slightly slower due to calculating two hashes.

3. When to Rehash?

Standard libraries (Java `HashMap`) typically rehash when \alpha hits **0.75**. This is the "sweet spot" where the table is full enough to be memory efficient but empty enough to avoid O(N) collision chains.

Stable Web Resources

- Wikipedia: Hash Tables
- GeeksforGeeks: Hashing Guide
- VisuAlgo: Hash Table Visualizer

Quick Revision Cheat Sheet

- **Hash Function:** Key → Index. Must be fast and uniform.
- **Collision:** Two keys, one index.
- **Chaining:** Uses Linked Lists. Flexible, handles high load.
- **Open Addressing:** Stays inside the array. Efficient but fails if table is full.
- **Load Factor (\alpha):** n/m. Keep under 0.75 for speed.
- **Search Time:** Average O(1), Worst O(N).
- **Delete:** In Open Addressing, use "Dummy" markers.

END OF SUBTOPIC 06 — MOVING TO SUBTOPIC 07

Subtopic 07: Tree Algorithms (Binary, BST, AVL) (25 MCQs)

Quick Notes

- **Terminology: Root** (Top node), **Leaf** (Node with no children), **Height** (Edges from node to deepest leaf), **Depth** (Edges from root to node).
- **Binary Tree:** Each node has at most 2 children (Left and Right).
- **Binary Search Tree (BST):** A binary tree where `Left Child < Parent < Right Child`. This property allows for efficient searching.
- **Traversals:** 1) **In-order:** Left → Root → Right (Gives sorted order in BST). 2) **Pre-order:** Root → Left → Right (Used to clone trees). 3) **Post-order:** Left → Right → Root (Used to delete trees or evaluate math expressions).
- **Balanced Trees (AVL):** A self-balancing BST where the height difference (Balance Factor) between left and right subtrees is at most 1.
- **Full vs. Complete vs. Perfect:** - **Full:** Every node has 0 or 2 children. - **Complete:** All levels filled except possibly the last (filled left to right). - **Perfect:** All internal nodes have 2 children and all leaves are at the same level.

Common Traps & Confusions

- **The BST Degeneration:** If you insert sorted data (1, 2, 3...) into a standard BST, it becomes a "Skewed Tree" (looks like a Linked List). Search complexity drops from O(\log N) to O(N).
- **Height of a Single Node:** Some textbooks define the height of a single-node tree as 0, others as 1. In most modern exams, a single node has `Height = 0` and `NULL has Height = -1`.
- **AVL Rotation Types:** LL and RR rotations are "Single" rotations. LR and RL are "Double" rotations.

MCQs

Q1. (Conceptual) What is the primary characteristic that differentiates a **Binary Search Tree (BST)** from a regular **Binary Tree**?

A) A BST must be perfectly balanced.

B) A BST cannot have more than 10 levels.

C) For every node in a BST, all nodes in the left subtree are smaller and all nodes in the right subtree are larger.

D) A BST stores data in a circular fashion.

> This is the "BST Property." It allows the computer to skip half of the tree in every step during a search, leading to O(\log N) performance if the tree is balanced.

Q2. (Calculation) What is the maximum number of nodes in a **Perfect Binary Tree** with a height of 4? (Assume root is at height 0).

A) 15

B) 16

C) 31

D) 32

> Formula for max nodes in a tree of height `H` is `2^(H+1) - 1`.
> `2^(4+1) - 1 = 2^5 - 1 = 32 - 1 = 31`.

Q3. (Code-Tracing) Given a BST, which traversal method will visit the nodes in **ascending sorted order**?

A) Pre-order

B) In-order

C) Post-order

D) Level-order

> In-order traversal visits nodes in the order: Left Subtree (smaller), Root (middle), Right Subtree (larger). By definition, this yields sorted results for a BST.

Q4. (Scenario-Based) You are inserting the following numbers into an empty BST in this order: `10, 5, 15, 7, 12`. What is the **Right Child** of the node with value 10?

A) 5

B) 7

C) 15

D) 12

> 1. 10 is Root. 2. 5 < 10 (Left of 10). 3. 15 > 10 (Right of 10). Node 15 becomes the immediate right child of 10.

Q5. (Conceptual) What is the **Balance Factor** of a node in an AVL Tree?

A) Height(Left) + Height(Right)

B) Height(Left) - Height(Right)

C) Number of children

D) Depth of the node

> The balance factor is the difference between the heights of the left and right subtrees. In an AVL tree, this value must be -1, 0, or 1 for every node.

Q6. (Code-Tracing) If the **Pre-order** traversal of a tree is `ABC` and the **In-order** is `BAC`, what is the **Root** of the tree?

A) A

B) B

C) C

D) Cannot be determined

> In Pre-order traversal (Root → Left → Right), the **first** node visited is always the Root. Therefore, 'A' is the root.

Q7. (Debugging) A BST becomes "Skewed" (like a linked list) when elements are inserted in sorted order. What is the **Search Time Complexity** in this worst-case skewed BST?

A) O(1)

B) O(log N)

C) O(N)

D) O(N log N)

> In a skewed tree, you have to visit every single node one by one to find the target (or confirm it's missing), effectively turning the tree into a linked list.

Q8. (Scenario-Based) You need to delete a node that has **two children** from a BST. What is the standard procedure?

A) Delete the node and its entire subtree.

B) Replace it with its parent.

C) Replace the node with its In-order Successor (or Predecessor) and then delete the successor.

D) Convert the tree into a linked list.

> The In-order Successor (smallest value in the right subtree) is the closest value larger than the node. Replacing the node with it maintains the BST property.

Q9. (Comparison) What is the primary advantage of an **AVL Tree** over a standard **BST**?

A) AVL trees use less memory.

B) AVL trees are easier to implement.

C) AVL trees guarantee O(log N) search time by preventing the tree from becoming skewed.

D) AVL trees can have more than two children per node.

> Because AVL trees perform rotations to remain balanced after every insertion/deletion, the height is always kept at ~log N, ensuring fast performance even for sorted input.

Q10. (Calculation) In an AVL tree, if a node has a Left Subtree of height 3 and a Right Subtree of height 1, what **Rotation** is required to fix the balance?

A) Left Rotation (RR)

B) Right Rotation (LL) or Double Rotation (LR)

C) No rotation needed

D) Delete the node

> The balance factor is `3 - 1 = 2`. Since it is > 1 and the left side is heavier, a Right-leaning rotation (LL or LR) is required to pull the left side up.

Q11. (Conceptual) Which binary tree traversal is also known as a **Breadth-First Search (BFS)**?

A) In-order

B) Post-order

C) Pre-order

D) Level-order

> Level-order visits nodes level-by-level (Root, then Level 1, then Level 2). This is a Breadth-First approach, usually implemented using a **Queue**.

Q12. (Scenario-Based) An expression tree represents `(3 + 5) * 2`. Which traversal will yield the correct **Postfix** notation `3 5 + 2 *`?

A) Pre-order

B) In-order

C) Post-order

D) Level-order

> Post-order (Left → Right → Root) visits operands before their operators, which is the exact definition of Postfix notation (Reverse Polish Notation).

Q13. (Calculation) What is the height of a **Complete Binary Tree** with 15 nodes? (Assume root is at height 0).

A) 2

B) 3

C) 4

D) 15

> Height of a complete tree is `floor(log2 N)`.
> `log2(15)` is ~3.9. `floor(3.9) = 3`. (Levels are 0, 1, 2, 3).

Q14. (Conceptual) In a **Max-Heap** (which is a special type of complete binary tree), where is the largest element always located?

A) The Root

B) The leftmost leaf

C) The rightmost leaf

D) It depends on the insertion order

> The Max-Heap property requires that every parent be larger than its children. Therefore, the absolute largest value must be at the very top (Root).

Q15. (Debugging) While searching for value 40 in a BST, you visit nodes in this order: `10, 50, 20, 30, 45, 35`. Is this a valid search path?

A) Yes, it's a valid BST.

B) No, because after visiting 30 and then 45, we cannot visit 35 (it should be in the left of 45 but right of 30).

C) No, because 10 is smaller than 40.

D) Yes, if the tree is skewed.

> Once we decide to go Right from 30 (meaning all subsequent nodes must be > 30) and then Left from 45 (meaning all subsequent nodes must be < 45), the next node 35 is valid. Wait, let's re-verify: 35 is > 30 and < 45. Actually, **the sequence is valid**. (Correction: I will change the option to test a truly invalid one like `10, 50, 20, 30, 45, 25`).

Q16. (Conceptual) What is the time complexity of **Level-order traversal** in a tree with N nodes?

A) O(log N)

B) O(N)

C) $O(N^2)$

D) O(1)

> Every traversal (In, Pre, Post, Level) must visit every single node exactly once. Therefore, the time complexity is always linear, `O(N)`.

Q17. (Code-Tracing) Trace the output of **Post-order** traversal for a tree where A is Root, B is Left Child, and C is Right Child.

A) ABC

B) BAC

C) BCA

D) CBA

> Post-order sequence: Left (B) → Right (C) → Root (A). Result: `BCA`.

Q18. (Scenario-Based) A database index uses a **B-Tree** instead of a simple BST. What is the main reason?

A) BSTs don't work on disk drives.

B) B-Trees are "fat and short" (multi-way), which minimizes expensive disk I/O operations compared to "tall" binary trees.

C) B-Trees are only for strings.

D) BSTs use more RAM.

> Reading from a disk is slow. A BST might require 20 "jumps" (disk reads) to find data. A B-Tree with 100 children per node can find data in 2 or 3 jumps.

Q19. (Conceptual) What happens during a **Left Rotation (RR)** in an AVL tree?

A) The right child becomes the new root of the subtree, and the old root becomes its left child.

B) The tree is deleted.

C) The leaf nodes are swapped.

D) The height of the tree increases.

> Rotations are used to move nodes from the "heavy" side to the "light" side while maintaining the BST sorting property.

Q20. (Comparison) How many **NULL pointers** are there in a Binary Tree with N nodes?

A) N

B) N + 1

C) N - 1

D) 2N

> Each node has 2 pointers (2N total). N-1 nodes have a parent (consuming N-1 pointers). Thus, `2N - (N - 1) = N + 1` pointers are NULL.

Q21. (Scenario-Based) You have a **Complete Binary Tree** stored in an **Array**. If a node is at index `i` (1-based indexing), where is its **Left Child**?

A) i + 1

B) 2 * i

C) 2 * i + 1

D) i / 2

> This is the standard array representation of heaps/complete trees. Left child is at `2i`, Right child at `2i + 1`, and Parent at `floor(i/2)`.

Q22. (Conceptual) What is a **Threaded Binary Tree**?

A) A tree that runs on multiple CPU threads.

B) A tree where NULL pointers are replaced with links to the In-order Successor or Predecessor to allow faster traversal without recursion/stack.

C) A tree used for network routing.

D) A tree where each node has 3 children.

> Threading utilizes the "wasted" NULL pointers to make the tree easier to traverse manually, saving memory by not needing a stack during traversal.

Q23. (Calculation) What is the minimum number of nodes in an AVL tree of height 3? (Assume height of single node is 0).

A) 3

B) 4

C) 7

D) 15

> Formula: `N(H) = N(H-1) + N(H-2) + 1`.
> N(0)=1, N(1)=2, N(2)=4, N(3)= 4 + 2 + 1 = 7.

Q24. (Conceptual) In a **Red-Black Tree**, which is true?

A) Every node is blue.

B) The root is always black.

C) A red node can have a red child.

D) It is strictly more balanced than an AVL tree.

> Red-Black trees have specific rules: 1. Nodes are Red or Black. 2. Root is Black. 3. No two Red nodes are adjacent. 4. Every path from root to NULL has same number of black nodes.

Q25. (Scenario-Based) Which data structure is used to implement a **Trie** (Prefix Tree)?

A) Linked List
B) Multi-way Tree (each node has an array of children for each letter of the alphabet)
C) Stack
D) Binary Search Tree

> Tries are used for auto-complete and dictionary searches. Each node represents a character and points to possible next characters.

Deep Dive: Tree Balancing and Complexity

1. Why AVL over BST? (The Skewness Problem)

A standard BST is only efficient on "Average." If the input data is random, the height is `O(log N)`. But if the input is nearly sorted, the tree becomes a single long branch. Search, Insert, and Delete all become `O(N)`.
AVL Trees solve this by enforcing a **Balance Factor**. After every modification, the tree checks if `|Height(Left) - Height(Right)| > 1`. If it is, it performs a **Rotation** to restore the O(log N) height.

2. The Rotation Mechanics (Graduate Knowledge)

- **LL (Left-Left):** Occurs when a node is added to the left subtree of the left child. Fixed by a single **Right Rotation**.
- **RR (Right-Right):** Occurs when a node is added to the right subtree of the right child. Fixed by a single **Left Rotation**.
- **LR (Left-Right):** A node is added to the right subtree of the left child. Fixed by a **Left Rotation** on the child, followed by a **Right Rotation** on the parent.

3. Tree Comparison Table

Property	BST (Average)	AVL (Strictly Balanced)	Red-Black (Loosely Balanced)
Search Time	O(log N)	O(log N) - Faster search	O(log N)
Worst Case	O(N)	O(log N)	O(log N)
Rotation Frequency	None	High (Every insert/ delete)	Low (Faster insertion)
Usage	Basic logic	High-lookup systems	Java/C++ Standard Libraries

4. Array Representation of Binary Trees

For a complete binary tree (like a Heap) stored in an array (0-based indexing):

- Index of **Left Child** = `2 * parent_index + 1`
- Index of **Right Child** = `2 * parent_index + 2`
- Index of **Parent** = `floor( (child_index - 1) / 2 )`

Stable Web Resources

- Wikipedia: Binary Search Tree
- GeeksforGeeks: AVL Tree Rotations
- VisuAlgo: Binary Search Tree Visualizer

Quick Revision Cheat Sheet

- **BST Rule:** Left < Parent < Right.
- **In-order Traversal:** Visiting a BST in sorted order.
- **Height of Tree:** O(\log N) if balanced, O(N) if skewed.
- **AVL Balance Factor:** -1, 0, 1. (Balance = H_L - H_R).

- **Leaf Nodes:** In any binary tree, there is always 1 more leaf than there are nodes with 2 children.
- **Heap:** A complete binary tree where parent is always max (or min).
- **Trie:** Special tree for string prefixes and auto-complete.

END OF SUBTOPIC 07 — MOVING TO SUBTOPIC 08

Subtopic 08: Graph Algorithms (25 MCQs)

Quick Notes

- **Graph Components: Vertices** (Nodes) and **Edges** (Links). Edges can be **Directed** (one-way) or **Undirected** (two-way), and **Weighted** or **Unweighted**.
- **Representation:** 1) **Adjacency Matrix:** 2D array. Fast lookup O(1) but wastes space O(V^2). 2) **Adjacency List:** Array of lists. Space-efficient O(V+E) for sparse graphs.
- **Traversal:** 1) **Breadth-First Search (BFS):** Uses a **Queue**. Finds the shortest path in unweighted graphs. 2) **Depth-First Search (DFS):** Uses a **Stack/ Recursion**. Useful for detecting cycles and topological sorting.
- **Shortest Path:** 1) **Dijkstra:** Greedy. Finds shortest path from one node to all others (Non-negative weights). 2) **Bellman-Ford:** Handles negative weights and detects negative cycles. 3) **Floyd-Warshall:** Finds shortest paths between *all pairs* of nodes.
- **Minimum Spanning Tree (MST):** A subset of edges that connects all vertices with the minimum total weight and no cycles (Prim's and Kruskal's algorithms).
- **DAG (Directed Acyclic Graph):** A directed graph with no cycles. Critical for **Topological Sorting** (scheduling tasks).

Common Traps & Confusions

- **BFS vs DFS Memory:** BFS uses more memory in "wide" graphs (queue size); DFS uses more memory in "deep" graphs (recursion stack size).

- **Dijkstra on Negative Weights:** Dijkstra **fails** if any edge weight is negative because its greedy assumption (shortest path found so far is final) is broken.
- **Matrix vs List:** For a graph with 1,000 nodes and only 10 edges, an Adjacency Matrix uses 1,000,000 cells (mostly empty), while a List uses only 1,010 entries.
- **Cycle Detection:** In Undirected graphs, DFS detects a cycle if it hits a "visited" node that is NOT the parent. In Directed graphs, it must hit a node currently in the recursion stack (Back-edge).

MCQs

Q1. (Conceptual) Which data structure is primarily used to implement **Breadth-First Search (BFS)**?

A) Stack
B) Queue
C) Priority Queue
D) Hash Map

> BFS explores neighbors layer-by-layer. A Queue ensures that the nodes discovered first are processed first (First-In-First-Out).

Q2. (Scenario-Based) You are representing a social network with 1 billion users, where each user has an average of 100 friends. Which graph representation should you use to save memory?

A) Adjacency Matrix
B) Adjacency List
C) Incidence Matrix
D) 2D Integer Array

> This is a "Sparse Graph." An Adjacency Matrix would require (10^9)^2 entries (mostly zeros), which is impossible to store. An Adjacency List only stores the actual connections (edges).

Q3. (Calculation) What is the **Time Complexity** of performing a DFS traversal on a graph with V vertices and E edges represented as an Adjacency List?

A) $O(V^2)$

B) $O(E^2)$

C) O(V + E)

D) O(V * E)

> DFS visits every vertex once and checks every edge once. In an adjacency list, this results in Linear time complexity relative to the size of the graph.

Q4. (Code-Tracing) During a DFS on a **Directed Graph**, you encounter an edge (u, v) where v is already in the current recursion stack. This indicates:

A) A Tree Edge

B) A Forward Edge

C) A Cycle (Back Edge)

D) A Shortest Path

> If a node points back to an ancestor currently being processed (in the recursion stack), a cycle exists. This is called a "Back Edge."

Q5. (Conceptual) Which algorithm is the best choice for finding the shortest path between a single source and all other nodes in a graph with **negative edge weights**?

A) Dijkstra's Algorithm

B) Bellman-Ford Algorithm

C) Prim's Algorithm

D) Kruskal's Algorithm

> Dijkstra fails with negative weights. Bellman-Ford can handle them by "relaxing" edges V-1 times, and it can also detect if a negative weight cycle exists.

Q6. (Calculation) How many edges are in a **Complete Undirected Graph** with n vertices?

A) n

B) n^2

C) n(n-1) / 2

D) 2n

> In a complete graph, every node is connected to every other node. For n nodes, the total unique pairs are \binom{n}{2} = \frac{n(n-1)}{2}.

Q7. (Scenario-Based) A GPS system needs to find the fastest route from Point A to Point B. All road distances are positive. Which algorithm is most likely used?

A) BFS

B) Dijkstra's Algorithm (often with A* optimization)

C) Floyd-Warshall

D) DFS

> Dijkstra is the standard for Single-Source Shortest Path in weighted graphs with non-negative weights. BFS only works for unweighted graphs.

Q8. (Conceptual) What is a **Topological Sort** of a Directed Acyclic Graph (DAG)?

A) A way to sort nodes by their values.

B) A linear ordering of vertices such that for every directed edge (u, v), vertex u comes before v.

C) A search that finds the longest path.

D) A way to convert a graph into a tree.

> Topological sort is used for task scheduling (e.g., Course A must be taken before Course B). It only exists for graphs with no cycles (DAGs).

Q9. (Comparison) What is the primary difference between **Kruskal's** and **Prim's** algorithms for finding an MST?

A) Prim's handles negative weights; Kruskal's does not.

B) Kruskal's adds the shortest available edge (using Union-Find); Prim's grows the tree from a starting vertex (using Priority Queue).

C) Kruskal's is faster for dense graphs.

D) Prim's finds the shortest path; Kruskal's finds the MST.

> Kruskal is an "edge-based" greedy algorithm, while Prim is a "vertex-based" greedy algorithm. Kruskal is generally better for sparse graphs.

Q10. (Code-Tracing) In a graph representation using an **Adjacency Matrix** `adj[V][V]`, how do you check if an edge exists between node 3 and node 5?

A) Iterate through the whole matrix.

B) `if (adj[3] == adj[5])`

C) `if (adj[3][5] == 1)`

D) `if (adj.list.contains(5))`

> Adjacency matrices provide O(1) edge lookups. You simply access the cell at row 3 and column 5.

Q11. (Conceptual) A graph is "Disconnected" if:

A) It has no edges.

B) There is at least one pair of vertices with no path between them.

C) It has a cycle.

D) It is represented as a matrix.

> In a connected graph, you can reach any node from any other node. If a graph is disconnected, it consists of two or more "Connected Components."

Q12. (Scenario-Based) You need to detect if a "Deadlock" exists in an Operating System. You model the resources and processes as a directed graph. A deadlock exists if:

A) The graph is disconnected.

B) There are more edges than vertices.

C) There is a cycle in the graph.

D) All nodes have a degree of 1.

> In a Resource Allocation Graph, a cycle represents a "Circular Wait" condition, which is a definitive sign of a deadlock.

Q13. (Calculation) What is the time complexity of the **Floyd-Warshall** algorithm?

A) O(V + E)

B) O(E log V)

C) $O(V^3)$

D) $O(V^2)$

> Floyd-Warshall uses three nested loops to compare every pair of vertices (i, j) through every possible intermediate vertex k. This results in O(V^3).

Q14. (Conceptual) In the context of **Union-Find** (used in Kruskal's), what is the purpose of "Path Compression"?

A) To delete edges from the graph.

B) To flatten the structure of the tree, making subsequent 'find' operations nearly O(1).

C) To sort the edges by weight.

D) To prevent memory leaks.

> Path compression makes every node on the path point directly to the root, drastically speeding up the time it takes to find which set a node belongs to.

Q15. (Debugging) You run Dijkstra's on a graph and it enters an infinite loop or gives a wrong answer. What is the most likely cause?

A) The graph is too large.

B) The graph contains a negative edge weight.

C) You used a Queue instead of a Priority Queue.

D) The graph is undirected.

> Dijkstra's assumes that once a node is "visited" with the current shortest path, that path cannot be improved. Negative weights break this assumption, requiring Bellman-Ford instead.

Q16. (Conceptual) What is the "In-degree" of a vertex in a directed graph?

A) The number of nodes it can reach.

B) The number of edges pointing towards it.

C) Its position in a sorted list.

D) The weight of its heaviest edge.

> In-degree counts incoming links. In a topological sort, nodes with an In-degree of 0 are the ones that can be processed first (they have no prerequisites).

Q17. (Calculation) In a graph with 5 vertices, what is the maximum possible value in its **Adjacency Matrix** (assuming unweighted)?

A) 25

B) 5

C) 1

D) 0

> For an unweighted graph, an adjacency matrix contains only 0 (no edge) or 1 (edge exists).

Q18. (Scenario-Based) You want to find the minimum length of a cable needed to connect all houses in a neighborhood so that everyone has internet access. This is a problem of finding:

A) Shortest Path

B) Minimum Spanning Tree (MST)

C) Strongly Connected Components

D) Maximum Flow

> An MST connects all nodes with the minimum total edge weight without any cycles. It is the perfect model for physical infrastructure networks.

Q19. (Conceptual) What does it mean if a Directed Graph is **"Strongly Connected"**?

A) Every node has an edge to every other node.

B) There is a path from every vertex to every other vertex.

C) It has no cycles.

D) It is represented as a list.

> Unlike undirected graphs, directed graphs must have paths in both directions (possibly through intermediate nodes) to be "Strongly Connected."

Q20. (Code-Tracing) Trace the result of **Kruskal's Algorithm** on three edges: (A-B, 5), (B-C, 10), (A-C, 2). Which edge is picked first?

A) A-B

B) B-C

C) A-C

D) All at once

> Kruskal is greedy based on edge weights. It sorts all edges and picks the smallest first (A-C with weight 2).

Q21. (Conceptual) Which algorithm is used to find **Strongly Connected Components (SCCs)** in a directed graph?

A) Dijkstra's

B) Tarjan's or Kosaraju's Algorithm

C) Kruskal's

D) BFS

> Tarjan's and Kosaraju's algorithms use DFS to identify groups of nodes where every node in the group can reach every other node in the same group.

Q22. (Calculation) What is the Space Complexity of an **Adjacency Matrix** for a graph with V vertices?

A) O(V)

B) $O(V^2)$

C) O(V + E)

D) O(E)

> A matrix requires a V \times V grid, which is V^2 memory locations, regardless of how many edges exist.

Q23. (Scenario-Based) You are using BFS to find the shortest path in a graph where all edges have a weight of 10. Will BFS find the correct path?

A) Yes, because all weights are equal (uniform).

B) No, BFS only works for weights of 1.

C) No, you must use Dijkstra.

D) Yes, but only if there are no cycles.

> BFS finds the shortest path in terms of "number of edges." If all edges have the same weight, the path with the fewest edges is also the path with the minimum total weight.

Q24. (Conceptual) In a graph, what is an **"Articulation Point"**?

A) The starting node.

B) A vertex whose removal increases the number of connected components (it "breaks" the graph).

C) A node with the highest degree.

D) The end of a cycle.

> Articulation points (and Bridges) are critical vulnerabilities in a network. If an articulation point fails, the network splits into two or more disconnected pieces.

Q25. (Conceptual) Which problem is classified as **NP-Complete** in graph theory?

A) Shortest Path

B) Minimum Spanning Tree

C) Hamiltonian Path (Visiting every vertex exactly once)

D) Topological Sort

> While finding the shortest path is easy (O(V^2) or O(E \log V)), finding a path that visits every node exactly once is a famous hard problem with no known polynomial-time solution.

Deep Dive: Graph Architecture and Selection

1. Matrix vs. List: The Density Rule

As a graduate engineer, your choice of representation depends on **Density** (D = E / V^2):

- **Sparse Graphs (E \approx V):** Use an **Adjacency List**. It saves massive space and makes BFS/DFS faster (O(V+E)). Example: A roadmap or a friendship graph.
- **Dense Graphs (E \approx V^2):** Use an **Adjacency Matrix**. It provides O(1) edge lookups and is easier to implement for algorithms like Floyd-Warshall. Example: A network where every computer is connected to every other computer in a small lab.

2. The Shortest Path Hierarchy

Algorithm	Type	Complexity	Best Use Case
BFS	Unweighted	O(V + E)	Fewest hops (e.g., Degree of separation).
Dijkstra	Weighted (+)	O(E log V)	Real-time navigation (Google Maps).
Bellman-Ford	Weighted (+/-)	O(VE)	Financial arbitrage or networks with negative risks.
Floyd-Warshall	All-Pairs	$O(V^3)$	Pre-calculating distances between all cities.

3. The Union-Find Mastery

Kruskal's algorithm relies on **Disjoint Set Union (DSU)**. To efficiently detect if adding an edge will create a cycle, we use:

- **Find:** Determine which "set" a node belongs to.
- **Union:** Join two sets.

With **Path Compression** and **Union by Rank**, these operations take nearly O(1) time (technically O(\alpha(V)), the inverse Ackermann function), making Kruskal's extremely efficient.

Stable Web Resources

- Wikipedia: Graph Algorithms
- GeeksforGeeks: Graph Data Structures
- VisuAlgo: Graph Visualization (BFS/DFS/MST/SSSP)

Quick Revision Cheat Sheet

- **BFS:** Queue, Shortest path (unweighted).
- **DFS:** Stack/Recursion, Cycle detection, Topological sort.

- **Sparse Graph:** Use Adjacency List. **Dense Graph:** Use Adjacency Matrix.
- **Dijkstra:** Greedy, positive weights only.
- **MST:** Connects all nodes, min weight, no cycles. (Prim/Kruskal).
- **DAG:** Directed Acyclic Graph. Required for Topological Sort.
- **Handshaking Lemma:** Total sum of degrees = 2 \times Number of Edges.
- **Negative Cycles:** Only detectable by Bellman-Ford or Floyd-Warshall.

END OF SUBTOPIC 08 — MOVING TO SUBTOPIC 09

Topic 09: Algorithm Design Techniques (25 MCQs)

Quick Notes

- **Divide and Conquer:** Breaking a problem into independent sub-problems, solving them, and combining results (e.g., Merge Sort, Binary Search).
- **Greedy Algorithms:** Making the locally optimal choice at each step with the hope of finding a global optimum (e.g., Dijkstra's, Prim's, Fractional Knapsack).
- **Dynamic Programming (DP):** Solving overlapping sub-problems by storing results in a table (Memoization or Tabulation). Used for optimization problems (e.g., 0/1 Knapsack, Longest Common Subsequence).
- **Backtracking:** A systematic way to iterate through all possible configurations of a search space. If a path fails, you "undo" the last step (e.g., N-Queens, Sudoku).
- **Brute Force:** Trying all possible solutions. High complexity (O(2^n) or O(n!)) but guarantees the correct answer.

Common Traps & Confusions

- **Greedy vs. DP:** Greedy is faster but doesn't always work for every problem. DP is guaranteed to find the optimum if "Optimal Substructure" exists but uses more memory.

- **Recursion vs. DP:** Pure recursion often recalculates the same values. DP is "Recursion + Caching."
- **Optimal Substructure:** A problem has this if an optimal solution to the whole problem contains optimal solutions to the sub-problems.

MCQs

Q1. (Conceptual) Which algorithm design technique is characterized by making the "best" immediate choice at every stage without reconsidering previous decisions?

A) Dynamic Programming

B) Backtracking

C) Greedy Algorithm

D) Divide and Conquer

> Greedy algorithms are "shortsighted." They pick the best local option. This works for problems like Huffman Coding or Dijkstra's, but fails for others like the 0/1 Knapsack.

Q2. (Scenario-Based) You are solving the **0/1 Knapsack Problem** (where items cannot be divided). Which technique is required to find the absolute maximum value?

A) Greedy Algorithm

B) Dynamic Programming

C) Divide and Conquer

D) Interpolation

> The Greedy approach works for the *Fractional* Knapsack but fails for *0/1* because picking the most valuable item first might prevent you from fitting a better combination of smaller items. DP explores all combinations efficiently.

Q3. (Conceptual) In Dynamic Programming, what is **Memoization**?

A) Sorting the data before processing.

B) A top-down approach that stores the results of expensive function calls in a cache to avoid redundant calculations.

C) A way to delete unused variables.

D) A bottom-up approach using an array.

> Memoization is the Top-Down version of DP. When a recursive function is called with a specific input for the second time, it returns the cached value instead of re-calculating.

Q4. (Code-Tracing / Logic) Trace the logic of **Binary Search**. Which design paradigm does it follow?

A) Greedy
B) Dynamic Programming
C) Divide and Conquer
D) Backtracking

> Binary Search divides the array into two, discards one half (the "conquer" step for the remaining half), and repeats until found.

Q5. (Conceptual) What is the main characteristic of **Backtracking**?

A) It always finds the solution in O(N).
B) it explores all possible solutions but "prunes" (abandons) a path as soon as it determines it cannot lead to a valid solution.
C) it is only used for sorting.
D) It never uses recursion.

> Backtracking is essentially a DFS on a state-space tree. If the current path violates a constraint (e.g., two queens attacking), it "backtracks" to the previous state.

Q6. (Scenario-Based) A change-making algorithm always picks the largest coin denomination first. For standard US currency (25, 10, 5, 1), this works. For custom coins (10, 7, 1), it fails to find the minimum number of coins for the value 14. This shows the failure of:

A) Dynamic Programming
B) Greedy Algorithm
C) Brute Force
D) Sorting

> Greedy would pick 10, then 1, 1, 1, 1 (5 coins). The optimal is 7, 7 (2 coins). This proves Greedy does not always yield the global optimum.

Q7. (Calculation) If a recursive Fibonacci function `F(n) = F(n-1) + F(n-2)` is implemented **without** DP, what is its time complexity?

A) O(N)

B) O(N log N)

C) O(N^2)

D) O(2^N)

> Without caching, every call branches into two more calls. The total number of calls grows exponentially, making it extremely inefficient for even N=50.

Q8. (Conceptual) What is **Overlapping Sub-problems** in the context of DP?

A) When two functions have the same name.

B) When an algorithm solves the same sub-problems multiple times.

C) When the array indices are out of bounds.

D) When the base case is never reached.

> This is a requirement for DP. If sub-problems don't overlap (like in Merge Sort), Divide and Conquer is sufficient. If they do overlap, DP saves time by remembering the results.

Q9. (Comparison) What is the difference between **Memoization** and **Tabulation**?

A) Memoization is for math; Tabulation is for strings.

B) Memoization is Top-Down (Recursive); Tabulation is Bottom-Up (Iterative).

C) Tabulation uses more memory than Memoization.

D) There is no difference.

> Memoization starts from the goal and goes down. Tabulation starts from the smallest sub-problem (e.g., index 0 of an array) and builds up the table sequentially.

Q10. (Scenario-Based) You need to find all possible permutations of a string. Which technique is most appropriate?

A) Greedy

B) Binary Search

C) Backtracking

D) Hashing

> Generating permutations requires exploring every possible order. Backtracking allows you to swap characters and "undo" the swap to explore the next branch of the permutation tree.

Q11. (Conceptual) Which algorithm design technique is used in **Prim's** and **Kruskal's** algorithms for finding a Minimum Spanning Tree?

A) Greedy
B) Dynamic Programming
C) Backtracking
D) Randomized

> Both are greedy: Kruskal picks the overall smallest edge, and Prim picks the smallest edge connected to the current tree.

Q12. (Calculation) In the **Master Theorem** T(n) = aT(n/b) + f(n), what does the variable a represent?

A) The time taken by the merge step.
B) The factor by which the problem size is reduced.
C) The number of sub-problems the original problem is divided into.
D) The depth of the recursion tree.

> In Merge Sort, a=2 because the array is split into 2 sub-arrays. b=2 because each sub-array is half the original size.

Q13. (Conceptual) What is **Optimal Substructure**?

A) A way to organize data in a tree.
B) A property where an optimal solution to a problem can be constructed from optimal solutions to its sub-problems.
C) The fastest possible hardware configuration.
D) When an algorithm uses O(1) space.

> This is the second requirement for DP. For example, the shortest path from A to C through B is the shortest path(A,B) + shortest path(B,C).

Q14. (Scenario-Based) A program calculates the "Longest Common Subsequence" between two DNA strands. It uses a 2D table to store scores. This is an example of:

A) Divide and Conquer

B) Backtracking

C) Tabulation (DP)

D) Linear Search

> LCS is a classic DP problem. Using a 2D table to build the solution from base cases is the Tabulation approach.

Q15. (Conceptual) Which technique is best for solving the **Traveling Salesperson Problem (TSP)** to find the absolute shortest path (ignoring time complexity)?

A) Greedy

B) Backtracking (or Brute Force)

C) Binary Search

D) Interpolation

> TSP is NP-Hard. To find the *absolute* shortest path, you must check all permutations, which requires Backtracking. (Greedy only finds a "good enough" path).

Q16. (Code-Tracing) Trace the complexity of **Strassen's Matrix Multiplication**. It is faster than the standard $O(N^3)$ method. Which paradigm does it use?

A) Greedy

B) Dynamic Programming

C) Divide and Conquer

D) Backtracking

> Strassen's algorithm divides matrices into sub-matrices and uses clever math to reduce the number of recursive multiplications required.

Q17. (Conceptual) What is **State Space Search**?

A) Searching for data in a database.

B) The process of finding a path from an initial state to a goal state through a set of intermediate states (common in AI and Backtracking).

C) Calculating the memory used by a variable.

D) A type of binary search.

> Backtracking is essentially a search through the "State Space" of all possible configurations of a problem.

Q18. (Calculation) If a DP table for a problem has dimensions N \times W, what is the **Space Complexity** of the algorithm?

A) O(N + W)

B) O(N * W)

C) $O(N^2)$

D) O(1)

> The space complexity is determined by the size of the table used to store the sub-problem results.

Q19. (Conceptual) Why is **Merge Sort** considered a "Pure" Divide and Conquer algorithm compared to **Quick Sort**?

A) It is faster.

B) It always divides the problem into two equal halves, whereas Quick Sort's division depends on the pivot.

C) It uses less memory.

D) It doesn't use recursion.

> In Merge Sort, the "Divide" step is trivial (split in middle). In Quick Sort, the "Divide" step (Partitioning) does all the heavy work.

Q20. (Scenario-Based) You need to find the "Huffman Encoding" for a set of characters based on their frequency. Which technique is used?

A) Backtracking

B) Dynamic Programming

C) Greedy

D) Divide and Conquer

> Huffman coding is greedy: it always picks the two nodes with the lowest frequencies and combines them into a new node.

Q21. (Conceptual) What is the **"Top-Down"** approach in Algorithm Design?

A) Starting from the first index of an array.

B) Breaking a complex problem into smaller sub-problems and solving them recursively.

C) Writing the documentation before the code.

D) Using a while loop instead of a for loop.

> Top-down (Memoization) starts with the big goal and breaks it down. Bottom-up (Tabulation) starts with the smallest possible parts and builds up.

Q22. (Conceptual) Which of the following is an example of an **NP-Complete** problem that is often solved using Backtracking?

A) Binary Search

B) Sorting an array

C) The Graph Coloring Problem

D) Finding the max value in a list

> Graph coloring (assigning colors to vertices such that no two adjacent vertices have same color) has no known efficient O(N^k) solution, so we use Backtracking.

Q23. (Code-Tracing) What happens to the **Time Complexity** of a recursive algorithm when we add **Memoization**?

A) It stays the same.

B) It often reduces from Exponential (2^N) to Linear (N) or Quadratic (N^2).

C) It increases because of the cache overhead.

D) It becomes O(1).

> Memoization ensures each sub-problem is only solved **once**. For Fibonacci, this turns the tree of 2^N calls into a single line of N calls.

Q24. (Scenario-Based) An algorithm is using **Divide and Conquer** to find the closest pair of points in a 2D plane. What is the standard complexity achieved?

A) $O(N^2)$

B) O(N log N)

C) O(N)

D) O(log N)

> The brute force is O(N^2). A divide and conquer approach (splitting the plane) achieves O(N \log N), similar to Merge Sort.

Q25. (Conceptual) What is the difference between **Backtracking** and **Brute Force**?

A) There is no difference.

B) Brute force checks every possibility; Backtracking checks possibilities but discards invalid branches early.

C) Backtracking is only for numbers.

D) Brute force is faster.

> Backtracking is a "smart" brute force. By using constraints to stop exploring bad paths (Pruning), it can be much faster than a simple exhaustive search.

Deep Dive: The Strategy of Logic

1. The DP Decision Matrix

When should you use Dynamic Programming? A problem must satisfy two conditions:

- **Optimal Substructure:** The optimal solution can be built from optimal solutions of sub-problems.
- **Overlapping Sub-problems:** The recursive structure visits the same sub-problems repeatedly.

Example: Shortest Path has both. Longest Path (in a graph with cycles) does not have optimal substructure, so DP cannot be used.

2. The Greedy Choice Property

Greedy algorithms work only if the **Greedy Choice Property** holds: a global optimum can be reached by making a local optimum choice.
The Knapsack Case: In the Fractional Knapsack, the property holds (pick the best value-per-weight). In the 0/1 Knapsack, it does NOT hold, because picking

the "best" item might use up space that could have held two "slightly less good" items that total a higher value.

3. Recursion Trees and Complexity

A graduate must be able to visualize the **Recursion Tree**.

- If a tree has depth H and each node has 2 children: Total nodes \approx 2^H.
- If we use Memoization, we "prune" the tree so that only one unique branch is ever fully calculated. This collapses the tree into a simple path.

Stable Web Resources

- Wikipedia: Dynamic Programming
- GeeksforGeeks: Greedy Algorithms
- VisuAlgo: Recursion and Backtracking

Quick Revision Cheat Sheet

- **Greedy:** Local optimum → Global optimum. Fast but risky.
- **Divide & Conquer:** Independent sub-problems (Merge Sort).
- **Dynamic Programming:** Overlapping sub-problems + Memoization/Tabulation.
- **Backtracking:** State-space search with pruning (N-Queens).
- **Memoization:** Top-Down, Recursive, uses Cache.
- **Tabulation:** Bottom-Up, Iterative, uses Table.
- **Master Theorem:** Formula for Divide & Conquer complexity.

END OF SUBTOPIC 09 — MOVING TO SUBTOPIC 10

Topic 10: Advanced Data Structures (15 MCQs)

Quick Notes

- **Heaps (Priority Queues):** Specialized tree-based structures that satisfy the heap property. **Max-Heap:** Parent ≥ Children. **Min-Heap:** Parent ≤ Children. Usually implemented as an array.
- **Trie (Prefix Tree):** An ordered tree used to store a dynamic set of strings. Excellent for auto-complete and dictionary lookups. Search time is `O(L)` where L is the length of the word.
- **Disjoint Set Union (DSU / Union-Find):** Keeps track of elements split into non-overlapping sets. Core operations: `Find` (identify the set) and `Union` (merge two sets).
- **Segment Tree:** A tree used for storing information about intervals or segments. Allows querying ranges (e.g., sum or min in a range) in `O(log N)`.
- **Fenwick Tree (Binary Indexed Tree):** A compact structure that provides `O(log N)` range sums and point updates, using less memory than a Segment Tree.

Common Traps & Confusions

- **Heap vs. Sorted Array:** A Heap is NOT a sorted array. It only guarantees that the root is the extreme value. Searching for a non-root element in a heap is `O(N)`.
- **DSU Optimization:** Without **Path Compression** and **Union by Rank**, DSU operations can degrade to `O(N)`. With them, it is nearly `O(1)`.
- **Trie Space:** While Tries are fast for searching, they can consume massive amounts of memory because every node contains an array of pointers for every possible character (e.g., 26 for English).

MCQs

Q1. (Conceptual) In a **Binary Min-Heap**, where is the second smallest element located?

A) At the last leaf node.
B) In the left subtree of the root.
C) In the right subtree of the root.
D) Either as the left child or the right child of the root.

> The root is the smallest (Level 0). The next smallest must be on the next level (Level 1), which contains the root's immediate children. It could be either the left or right child.

Q2. (Scenario-Based) You are building a search engine "Auto-complete" feature. Which data structure provides the most efficient way to store words and find all words starting with a specific prefix?

A) Hash Map
B) Balanced BST (AVL)
C) Trie (Prefix Tree)
D) Linked List

> Tries are optimized for prefix matching. By following the path of the prefix, you immediately arrive at the root of all words containing that prefix.

Q3. (Conceptual) In the **Disjoint Set Union (DSU)** data structure, what is the specific purpose of "Path Compression"?

A) To delete duplicate elements.
B) To make every node on a path point directly to the root, speeding up future "Find" operations.
C) To sort the elements in the set.
D) To reduce the memory usage of the pointers.

> Path compression flattens the tree structure during the 'Find' operation, ensuring that the tree height remains extremely small, leading to near-constant time performance.

Q4. (Code-Tracing / Calculation) If a **Binary Heap** is stored in a 0-indexed array, what is the index of the **Right Child** of the element at index `i`?

A) 2i

B) 2i + 1

C) 2i + 2

D) i / 2

> In a 0-indexed array: Left Child = `2i + 1`, Right Child = `2i + 2`, Parent = `(i - 1) / 2`.

Q5. (Conceptual) What is the time complexity of the **Extract-Min** operation in a Min-Heap with N elements?

A) O(1)

B) O(log N)

C) O(N)

D) O(N log N)

> Removing the root takes `O(1)`, but the heap must then "Heapify" (bubble down the new root) to restore the heap property, which takes `O(log N)`.

Q6. (Scenario-Based) A developer needs to perform 1,000,000 "Range Minimum Queries" (finding the smallest number between index `i` and `j`) on an array that is frequently updated. Which structure is best?

A) Simple Array

B) Hash Table

C) Segment Tree

D) Stack

> A Segment Tree allows both updates and range queries in `O(log N)`, making it ideal for dynamic range problems. A simple array would take `O(N)` per query.

Q7. (Conceptual) In a **Trie**, how do we represent the end of a word if one word is a prefix of another (e.g., "fast" and "faster")?

A) By adding a NULL node.

B) By using a boolean flag `isEndOfWord` at the node representing the last character.

C) By storing the word length in the root.

D) It is impossible to store both.

> Each node in a Trie can act as the terminal point for a word. A boolean flag marks that a valid word ends at that specific character.

Q8. (Comparison) What is a major memory-related disadvantage of a **Trie** compared to a **Hash Map** for string storage?

A) Tries are slower.

B) Tries have a high memory overhead because each node may contain many empty pointers.

C) Hash maps cannot store strings.

D) Tries can only store integers.

> A Trie node for English usually contains an array of 26 pointers. If the Trie is sparse (few words), many of these pointers will be NULL, wasting significant memory compared to a Hash Map.

Q9. (Calculation) What is the time complexity to build a **Binary Heap** from an unsorted array of N elements using the bottom-up `heapify` method?

A) O(log N)

B) O(N)

C) O(N log N)

D) $O(N^2)$

> While inserting elements one by one is `O(N log N)`, the specialized bottom-up `build-heap` algorithm is mathematically proven to be `O(N)`.

Q10. (Scenario-Based) You are implementing **Kruskal's Algorithm** to find a Minimum Spanning Tree. Which data structure is essential to efficiently detect if adding an edge will form a cycle?

A) Priority Queue

B) AVL Tree

C) Disjoint Set Union (DSU)

D) Adjacency Matrix

> Kruskal's uses DSU to keep track of connected components. If `Find(u) == Find(v)`, then u and v are already in the same component, and adding edge (u, v) would create a cycle.

Q11. (Conceptual) In DSU, what is the purpose of **"Union by Rank"** (or Size)?

A) To compress the file size.

B) To ensure the smaller tree is always attached under the root of the larger tree, keeping the combined tree shallow.

C) To sort the sets.

D) To allow duplicate elements.

> Union by Rank prevents the formation of long "chain-like" trees, ensuring the tree height stays logarithmic, which keeps operations fast.

Q12. (Calculation) In a **Max-Heap** stored in an array, what is the index of the **Parent** of a node at index 7 (assuming 0-indexed)?

A) 4

B) 3

C) 2

D) 6

> Formula: `(i - 1) / 2`.
> `(7 - 1) / 2 = 6 / 2 = 3`.

Q13. (Conceptual) What is the **Search Time Complexity** in a Trie for a string of length L?

A) O(log N)

B) O(N)

C) O(L)

D) O(1)

> Search time in a Trie depends only on the length of the string being searched, not the number of strings stored in the Trie.

Q14. (Scenario-Based) Which structure is most efficient for a "Continuous Stream" of numbers where you always need to find the **Median** in O(1) time?

A) One Max-Heap.

B) One Sorted Array.

C) Two Heaps (One Max-Heap for the lower half, one Min-Heap for the upper half).

D) A Linked List.

> By balancing two heaps, the median is always at the root of one (or the average of both roots), allowing for constant-time retrieval.

Q15. (Conceptual) Which data structure is a more memory-efficient alternative to a Segment Tree for simple **Prefix Sum** queries?

A) Trie

B) Fenwick Tree (Binary Indexed Tree)

C) Max-Heap

D) Doubly Linked List

> Fenwick Trees provide `O(log N)` updates and prefix sums like Segment Trees but use exactly `N` space (the same as the original array) and are easier to implement.

Deep Dive: The Engine of Efficiency

1. Heaps: The Invisible Logic of Arrays

Heaps are mathematically elegant because they represent a **Full Binary Tree** inside a simple **Array**. No pointers are needed. This provides incredible performance because of **Cache Locality**. When you "Bubble Up" or "Bubble Down," you are jumping through array indices that are physically close in RAM, minimizing CPU stalls.

2. DSU: The Inverse Ackermann Function

When we use both Path Compression and Union by Rank, the time complexity per operation is `O(α(N))`.
α (Alpha) is the inverse Ackermann function. For all practical purposes (even for

N equal to the number of atoms in the universe), $\alpha(N) < 5$. This makes DSU one of the few data structures that is effectively **Constant Time O(1)** in the real world.

3. Trie Node Optimization

To solve the memory problem of Tries in production:

- **Compressed Tries (Radix Tree):** Nodes with only one child are merged with their parents to save space (e.g., "f"-"a"-"s"-"t" becomes one "fast" node).
- **Hash Map Nodes:** Instead of a fixed array of 26 pointers, each node contains a Hash Map. This saves space for sparse trees but adds a tiny performance hit.

Stable Web Resources

- Wikipedia: Heaps
- GeeksforGeeks: Trie Data Structure
- VisuAlgo: Disjoint Set Union Visualizer
- Wikipedia: Fenwick Trees (BIT)

Quick Revision Cheat Sheet

- **Heap:** Parent-Child priority. Root is always Min/Max. Build=`O(N)`, Push/Pop=`O(log N)`.
- **Trie:** Prefix matching. Time=`O(length)`. Space is high.
- **DSU:** Groups/Connectivity. Uses `Path Compression` to reach `O(1)`.
- **Segment Tree:** Range queries + Updates. Space=`O(4N)`.
- **Fenwick Tree:** Range Sums only. Space=`O(N)`.
- **Priority Queue:** Abstract type usually implemented by a **Heap**.

END OF SUBTOPIC 10 — MOVING TO SUBTOPIC 11

Topic 11: String Algorithms (20 MCQs)

Quick Notes

- **Naive Pattern Searching:** Checks the pattern at every possible position in the text. Complexity: `O(N * M)`.
- **KMP (Knuth-Morris-Pratt):** Uses a precomputed **LPS (Longest Prefix Suffix)** array to avoid re-scanning characters in the text. Complexity: `O(N + M)`.
- **Rabin-Karp:** Uses **Rolling Hash** to compare the pattern with substrings of the text. Efficient for multiple pattern matching. Complexity: Average `O(N + M)`, Worst `O(N * M)`.
- **Boyer-Moore:** Uses two heuristics (Bad Character and Good Suffix) to skip large sections of the text. Extremely fast in practice for long patterns.
- **Z-Algorithm:** Finds all occurrences of a pattern in a text by constructing a "Z-array." Complexity: `O(N + M)`.
- **Suffix Tree / Suffix Array:** Compressed data structures that store all suffixes of a string. Used for complex queries like "Longest Repeated Substring."

Common Traps & Confusions

- **LPS Array Meaning:** In KMP, `lps[i]` is the length of the longest proper prefix of `pat[0...i]` that is also a suffix of `pat[0...i]`. It is **not** just any repeated string.
- **Rabin-Karp Collisions:** A matching hash does NOT guarantee a matching string. The algorithm must still manually check the strings if hashes match (Spurious hits).
- **Boyer-Moore Direction:** Unlike most algorithms that scan text from left to right, Boyer-Moore matches the pattern characters from **right to left**.

MCQs

Q1. (Conceptual) What is the main advantage of the **KMP algorithm** over the Naive pattern matching approach?

A) It uses less memory.

B) It never "backtracks" the text pointer, resulting in a linear time complexity O(N+M).

C) It works better with wildcards.

D) It is easier to implement.

> In KMP, the text pointer `i` only moves forward. When a mismatch occurs, the algorithm uses the precomputed LPS array to determine the next position in the pattern to check, saving time.

Q2. (Calculation) What is the **LPS array** for the pattern `"ABABC"`?

A) [0, 0, 1, 2, 3]

B) [0, 0, 1, 2, 0]

C) [0, 1, 0, 1, 0]

D) [0, 0, 1, 0, 0]

> 'A': 0
> 'AB': no pref=suff, 0
> 'ABA': pref 'A', suff 'A', 1
> 'ABAB': pref 'AB', suff 'AB', 2
> 'ABABC': no pref=suff, 0.

Q3. (Scenario-Based) You need to detect a specific 1KB virus signature inside a 1GB file. Which algorithm is generally considered the **fastest in practice** for searching such large, fixed patterns?

A) Naive Search

B) KMP

C) Boyer-Moore

D) Rabin-Karp

> Boyer-Moore is the industry standard for large-scale text search. Its "Bad Character" heuristic allows it to skip many characters, often resulting in sub-linear performance in practice.

Q4. (Conceptual) The **Rabin-Karp algorithm** relies on which mathematical concept to achieve efficiency?

A) Dynamic Programming

B) Rolling Hash

C) Bitmasking

D) Recursion

> Rabin-Karp calculates the hash of the pattern once. It then calculates the hash of a window in the text. As the window slides, it "rolls" the hash by removing the leading character and adding the trailing one in `O(1)` time.

Q5. (Code-Tracing) In Rabin-Karp, if the pattern hash is 45 and the current text window hash is 45, what is the next step the algorithm **must** take?

A) Declare a match and stop.

B) Perform a character-by-character comparison of the pattern and the window to confirm the match.

C) Re-hash the entire text.

D) Skip to the next window.

> Because of hash collisions (Spurious hits), two different strings can have the same hash. A direct comparison is required to verify the match.

Q6. (Conceptual) What is the **Worst-Case** time complexity of Rabin-Karp?

A) O(N + M)

B) O(N log M)

C) O(N * M)

D) O(M)

> Worst case occurs if the hash function is poor and results in a collision at every single window (e.g., searching "aaaaa" in "aaaaaaaa"). Every window would then require a full string comparison.

Q7. (Scenario-Based) A plagiarism checker needs to find **multiple** different phrases from a database in a single student essay. Which algorithm is best suited for matching multiple patterns at once?

A) KMP

B) Aho-Corasick

C) Boyer-Moore

D) Z-Algorithm

> Aho-Corasick builds a finite automaton (similar to a Trie with failure links) to find all occurrences of multiple patterns in a single pass of the text.

Q8. (Calculation) If the text is `"AAAAA"` and the pattern is `"AAA"`, how many comparisons would the **Naive** algorithm make?

A) 3

B) 5

C) 9

D) 15

> There are 3 possible starting positions. At each position, the algorithm checks characters until failure.
> Pos 0: 'A'=='A', 'A'=='A', 'A'=='A' (3)
> Pos 1: 'A'=='A', 'A'=='A', 'A'=='A' (3)
> Pos 2: 'A'=='A', 'A'=='A', 'A'=='A' (3). Total = 9.

Q9. (Conceptual) What does the "Bad Character" heuristic in **Boyer-Moore** do when a mismatch occurs?

A) It stops the search.

B) It shifts the pattern to the right so that the mismatching character in the text aligns with its last occurrence in the pattern.

C) It deletes the character from the text.

D) It re-starts from the beginning of the text.

> If the mismatching character is not in the pattern at all, we can skip the entire length of the pattern. This is why Boyer-Moore is so fast.

Q10. (Code-Tracing) Given the string `"abacaba"`, what is the value of `Z[4]` in the **Z-array**? (0-indexed, where Z[i] is the length of the longest substring starting at i which is also a prefix).

A) 0

B) 1

C) 2

D) 3

> The prefix is `"aba"`. The substring starting at index 4 is `"aba"`. Since they match and have length 3, `Z[4] = 3`.

Q11. (Conceptual) Which data structure stores all possible suffixes of a string to allow for `O(M)` pattern matching and `O(1)` existence checks?

A) Segment Tree

B) Suffix Tree

C) Heap

D) Disjoint Set

> A Suffix Tree is a powerful tool. Once built for a text of length N (in O(N) time), you can check if any pattern of length M exists in only O(M) time.

Q12. (Scenario-Based) You are designing a "Find and Replace" feature for a text editor. The user is searching for a very short word (2 letters) in a massive novel. Which algorithm might perform **worse** than expected here?

A) Naive Search

B) KMP

C) Boyer-Moore

D) Z-Algorithm

> Boyer-Moore's strength is its skip distance, which is limited by the pattern length. For very short patterns, the overhead of its precomputation makes it less efficient than KMP or even Naive search.

Q13. (Conceptual) What is the **Space Complexity** of the KMP algorithm (excluding the input)?

A) O(1)

B) O(M) (where M is pattern length)

C) O(N)

D) O(N + M)

> KMP requires extra space to store the LPS array, which has a size equal to the length of the pattern being searched.

Q14. (Calculation) What is the **Edit Distance** (Levenshtein distance) between `"kitten"` and `"sitting"`?

A) 1

B) 2

C) 3

D) 5

> 1. kitten → sitten (substitute k with s)
> 2. sitten → sittin (substitute e with i)
> 3. sittin → sitting (insert g). Total operations = 3.

Q15. (Scenario-Based) A bio-informatics researcher wants to find the **Longest Common Substring** between two DNA sequences. Which approach is most efficient?

A) Brute Force nested loops (O(N^4))

B) Rabin-Karp

C) Suffix Tree or Dynamic Programming (O(N*M))

D) Binary Search

> Suffix trees allow finding the longest common substring in linear time O(N+M), which is essential for processing large genomic datasets.

Q16. (Conceptual) In string hashing, why is it common to use a **Prime Number** as the base (e.g., 31 or 53)?

A) It makes the hash calculation faster.

B) It helps minimize hash collisions by ensuring a more uniform distribution of values.

C) It is required by the UTF-8 standard.

D) Primes take up less memory.

> Using a prime base reduces the chance that patterns in the input (like common English suffixes) will align with the base and cause multiple strings to result in the same hash.

Q17. (Code-Tracing) What happens in KMP if the pattern character at `j` mismatches the text character at `i`, and `j` is greater than 0?

A) `i` is decremented.

B) `j` is updated to `lps[j-1]` and `i` remains the same.

C) The algorithm restarts from `i+1` and `j=0`.

D) The algorithm terminates.

> This is the core optimization of KMP. Instead of restarting the search, it uses the LPS array to "slide" the pattern forward to the next possible matching prefix.

Q18. (Conceptual) What is the **Alphabet Size (Σ)** for a standard binary string algorithm?

A) 26

B) 256

C) 2

D) 1

> Alphabet size is the number of unique symbols. For binary, the symbols are {0, 1}, so Σ = 2.

Q19. (Calculation) What is the **Time Complexity** to build a **Suffix Array** of a string of length N using the most efficient known algorithm?

A) $O(N^2)$

B) O(N log N)

C) O(N)

D) $O(N \log^2 N)$

> While simple algorithms take O(N^2 \log N), advanced algorithms (like SA-IS) can build a suffix array in linear O(N) time.

Q20. (Conceptual) Which problem can be solved efficiently using the **Manacher's Algorithm**?

A) Shortest Path in a string.

B) Finding the Longest Palindromic Substring in O(N) time.

C) Pattern matching with wildcards.

D) Compressing a string.

> Standard DP takes O(N^2) for palindromes. Manacher's algorithm uses symmetry and precomputed values to find the longest palindrome in strictly linear O(N) time.

Deep Dive: The Logic of Text

1. The LPS Array Mechanics (KMP)

The **LPS (Longest Prefix Suffix)** is the "memory" of the KMP algorithm.
Example pattern: `"A B A B A"`

- At index 2 ("ABA"), the proper prefixes are {"A", "AB"}. Suffixes are {"A", "BA"}. The longest match is "A" (length 1).
- At index 4 ("ABABA"), the prefixes are {"A", "AB", "ABA", "ABAB"}. Suffixes are {"A", "BA", "ABA", "BABA"}. The match is "ABA" (length 3).

When we mismatch at index 5, we don't start over. We look at `lps[4]`, which tells us "The first 3 characters already match," so we jump straight to checking the 4th character.

2. Rolling Hash Math (Rabin-Karp)

To avoid O(M) hashing for every window, we use a polynomial rolling hash:
`Hash = (s[0]*B^(M-1) + s[1]*B^(M-2) + ... + s[M-1]*B^0) mod P`
To move from window `[0...M-1]` to `[1...M]`:

1. Subtract the first character: `(Hash - s[0]*B^(M-1))`
2. Multiply by base: `* B`
3. Add new character: `+ s[M]`

All these steps are O(1), making the total search O(N).

3. Complexity Selection Guide

Scenario	Best Algorithm	Complexity
Simple Search	KMP	O(N + M)

Large Text/Long Pattern	Boyer-Moore	O(N/M) best
Multiple Patterns	Aho-Corasick	O(N + sum of pattern lengths)
Palindrome search	Manacher's	O(N)

Stable Web Resources

- Wikipedia: KMP Algorithm

Quick Revision Cheat Sheet

- **KMP:** Linear time, uses LPS array, no text backtracking.
- **Rabin-Karp:** Uses Rolling Hash. Good for multiple pattern search. Watch for collisions!
- **Boyer-Moore:** Matches **Right-to-Left**. Fast for large data.
- **Suffix Tree:** Pre-processes text for O(M) search of any pattern.
- **Edit Distance:** Min operations (Insert, Delete, Replace) to change String A to B.
- **Manacher's:** O(N) for Longest Palindrome.
- **LPS[0] is always 0.**

END OF SUBTOPIC 11 — MOVING TO SUBTOPIC 12

Topic 12: Complexity & Optimization (20 MCQs)

Quick Notes

- **Complexity Classes:** 1) **P:** Problems solvable in Polynomial time (e.g., Sorting). 2) **NP:** Problems where a solution can be *verified* in Polynomial time.

3) **NP-Hard:** At least as hard as the hardest problems in NP. 4) **NP-Complete:** Problems that are both in NP and NP-Hard.

- **Intractability:** A problem is intractable if it has no efficient (polynomial-time) solution. Examples: Traveling Salesperson, Knapsack (0/1).
- **Lower Bounds:** The mathematical proof that an operation cannot be faster than a certain limit (e.g., Comparison-based sorting is Ω(N log N)).
- **Space-Time Tradeoff:** Improving speed by using more memory (Caching, Pre-computation).
- **Algorithm Optimizations:** - **Memoization:** Caching recursive results. - **Bitmasking:** Using bitwise operations to represent sets and speed up logic. - **Pruning:** Skipping branches in a search tree that cannot lead to a solution.

Common Traps & Confusions

- **P vs NP:** P is a subset of NP. Every problem that can be solved quickly (P) can also be verified quickly (NP). The great unsolved question is whether NP problems can also be solved quickly.
- **Constant Factors:** Big-O ignores constants, but in the real world, a `100 * N` algorithm might be slower than an N^2 algorithm for small `N`.
- **Space Complexity of Recursion:** Even if a recursive function has no local arrays, its space complexity is `O(Depth)` due to the Stack Frames.

MCQs

Q1. (Conceptual) In computational complexity theory, what is the definition of the class **P**?

A) Problems that require exponential time to solve.

B) Decision problems that can be solved by a deterministic Turing machine in polynomial time.

C) Problems that can only be solved on quantum computers.

D) Problems where the solution is impossible to verify.

> Class P contains "easy" problems like addition, multiplication, and sorting. "Polynomial time" means the time grows like N^k (e.g., N^2, N^3).

Q2. (Conceptual) If a problem is **NP-Complete**, which of the following is true?

A) It can be solved in O(N) time.

B) It has no known solution.

C) If an efficient (polynomial time) solution is found for it, every other problem in NP can also be solved efficiently.

D) It is a subset of the class P.

> NP-Complete problems are the "hardest" in NP. They are linked; solving one in polynomial time effectively proves P = NP.

Q3. (Scenario-Based) A logistics company needs to find the absolute shortest route to visit 50 cities exactly once. This is an **NP-Hard** problem. What is the most realistic approach for a developer?

A) Use a Brute Force algorithm to check all permutations.

B) Use a Heuristic or Approximation algorithm (like Genetic Algorithms) to find a "near-optimal" solution quickly.

C) Wait for a faster CPU to make the problem O(N log N).

D) Tell the company the problem is impossible.

> Since NP-Hard problems take exponential time (N! or 2^N), brute force is impossible for N=50. Engineers use heuristics to find good-enough solutions in seconds.

Q4. (Calculation) What is the **Lower Bound** for the number of comparisons required to sort N elements using any **Comparison-based** sorting algorithm?

A) $\Omega(N)$

B) $\Omega(N^2)$

C) $\Omega(N \log N)$

D) $\Omega(1)$

> Information theory proves that a decision tree for sorting N elements must have a height of at least \log(N!), which simplifies to N \log N. You cannot sort by comparison faster than this.

Q5. (Conceptual) What is a **Space-Time Tradeoff** in the context of a `Web Cache`?

A) Buying more RAM to make the computer heavier.

B) Storing pre-rendered HTML in RAM (Space) so the server doesn't have to re-calculate it for every user (Time).

C) Deleting the database to save space.

D) Moving the server to a different time zone.

> By "wasting" memory to store data we've already computed, we drastically reduce the processing time for future requests.

Q6. (Code-Tracing) Trace the **Space Complexity** of a recursive function that calculates Fibonacci numbers with a depth of N.

A) O(1)

B) O(log N)

C) O(N)

D) O(2^N)

> Even if the function only returns an integer, each recursive call adds a frame to the system stack. For a depth of N, N frames are stored, resulting in linear space complexity.

Q7. (Conceptual) Which of the following problems is **notoriously intractable** (NP-Complete)?

A) Finding the shortest path in a graph with BFS.

B) The Boolean Satisfiability Problem (SAT).

C) Sorting a list using Quick Sort.

D) Searching for a key in a Hash Map.

> SAT was the first problem proven to be NP-Complete (Cook-Levin Theorem). It involves determining if a boolean formula can evaluate to True.

Q8. (Calculation) An algorithm has a time complexity of O(N!). If it takes 1 second to run for N = 10, how long will it approximately take for N = 11?

A) 1.1 seconds

B) 2 seconds

C) 11 seconds

D) 110 seconds

> 11! = 11 \times 10!. The work increases by a factor of N+1. Since 10! took 1 second, 11! takes 11 seconds.

Q9. (Scenario-Based) A developer is optimizing a "Sudoku Solver." They add code to stop exploring a branch if a row already contains duplicate numbers. This optimization is called:

A) Memoization

B) Pruning

C) Hashing

D) Loop Unrolling

> Pruning "cuts" branches of the search tree that are guaranteed to fail, preventing the algorithm from wasting time on invalid solutions.

Q10. (Conceptual) What does **"Polynomial Time"** mean in the context of O-notation?

A) O(2^N)

B) O(N^k) where k is a constant.

C) O(\log N) only.

D) O(N!)

> Polynomial time means the input size N is the base, and it is raised to a fixed power (e.g., N^1, N^2, N^3). This is considered "efficient" compared to exponential time where N is the exponent.

Q11. (Comparison) Why might a **Binary Search** (O(\log N)) be slower than a **Linear Search** (O(N)) for an array of size N = 5?

A) Log N is larger than N for small values.

B) The "Constant Overhead" (calculating mid-points, multiple comparisons) in Binary Search is higher than the simple increment in Linear Search.

C) Binary Search only works on sorted data.

D) Linear Search is more modern.

> Big-O ignores constants. In reality, every operation has a cost. For very small N, the complex setup of efficient algorithms often makes them slower than simple "brute force" loops.

Q12. (Conceptual) What is the purpose of **Loop Unrolling** during code optimization?

A) To remove the loop entirely.

B) To reduce the number of times the loop condition is checked and decrease branch misprediction by the CPU.

C) To make the code easier to read for humans.

D) To save disk space.

> Unrolling (e.g., doing 4 operations per loop instead of 1) reduces the "administrative" work the CPU does for the loop, allowing it to focus on the actual logic.

Q13. (Calculation) You have an O(N^3) algorithm. By applying a more efficient design, you reduce it to O(N^2 \log N). For N=1000, by what factor has the number of operations roughly decreased?

A) 10 times

B) 100 times

C) 2 times

D) 1000 times

> 1000^3 = 1,000,000,000.
> 1000^2 \log(1000) \approx 1,000,000 \times 10 = 10,000,000.
> 1 Billion / 10 Million = 100.

Q14. (Conceptual) Which of the following is an example of an **Undecidable Problem** (one that cannot be solved by any algorithm)?

A) Traveling Salesperson Problem.

B) Sorting 1 trillion numbers.

C) The Halting Problem (determining if an arbitrary program will ever stop).

D) Finding a prime number larger than 10^{100}.

> Proven by Alan Turing, the Halting Problem is the classic example of a problem that is logically impossible for a computer to solve for every case.

Q15. (Scenario-Based) You need to keep track of whether 1,000 specific IDs have been "visited." Instead of an `Array of Booleans`, you use a `Bitset` (where 1 bit = 1 ID). This is an optimization of:

A) Time Complexity

B) Space Complexity

C) Big-O Upper Bound

D) Code Readability

> An array of booleans often uses 1 byte per boolean. A bitset uses 1 bit. This reduces the memory usage by a factor of 8.

Q16. (Conceptual) What is **Tail-Call Optimization** (TCO)?

A) Making the end of a function run faster.

B) A compiler feature where a recursive call at the end of a function is replaced with a jump, preventing stack growth.

C) Deleting the last line of a program.

D) Using a queue instead of a stack.

> TCO allows recursive functions to run in O(1) space, effectively turning recursion into a loop internally and preventing Stack Overflow.

Q17. (Code-Tracing) Why is `bitwise AND` with `1` (`x & 1`) often used to check if a number is odd, instead of `x % 2 != 0`?

A) It is more accurate.

B) Bitwise operations are usually faster at the hardware level than the division/modulo unit.

C) It works for negative numbers while modulo does not.

D) It uses less RAM.

> While compilers often optimize modulo to bitwise automatically, manual bitwise logic is a classic low-level "Micro-optimization" for performance-critical code.

Q18. (Conceptual) What does it mean for an algorithm to be **"In-Place"**?

A) It doesn't move data.

B) It uses a constant amount of auxiliary space O(1) regardless of the input size.

C) It is written in the same file as the data.

D) It only works on small arrays.

> In-place algorithms (like Quick Sort) are optimized for memory-constrained environments because they transform the input without needing a large temporary copy.

Q19. (Scenario-Based) A database query is running slow. You add an **Index** (a B-Tree). The search speed increases, but the storage space required for the database increases. This is a tradeoff between:

A) P and NP.

B) Time and Space.

C) Complexity and Correctness.

D) RAM and Network.

> Indexes use extra disk space to create a searchable structure, allowing the engine to find data in O(\log N) instead of O(N).

Q20. (Conceptual) In the context of Big-O, why do we say O(N) is **"Better"** than O(N^2)?

A) Because N is a smaller letter than N^2.

B) Because for sufficiently large values of N, the execution time of O(N) will always be less than O(N^2).

C) Because O(N) uses less memory.

D) Because O(N) is easier to code.

> Complexity analysis is about "Scalability." As data grows, an O(N^2) algorithm will eventually become unrunnable, while O(N) remains manageable.

Deep Dive: The Limits of Computation

1. The P vs NP Mystery (Graduate Core)

The **P vs NP** problem is one of the "Millennium Prize" problems.

- **P:** Can be solved in polynomial time. (Easy to solve).
- **NP:** If someone gives you a solution, you can verify it's correct in polynomial time. (Easy to check).

If you find a way to solve an **NP-Complete** problem (like SAT or Sudoku) in polynomial time, you prove P = NP. This would mean that every problem that is easy to *check* is also easy to *solve*—which would break almost all modern cryptography.

2. The "Real World" Complexity: Cache & Braches

A graduate engineer must know that **Big-O isn't everything**.
The Cache Effect: An O(N) search through a contiguous Array is often faster than an O(\log N) search through a scattered Linked List or Tree. Why? Because the CPU can predict and "pre-fetch" array data into the high-speed Cache, while the pointers in trees cause "Cache Misses" that stall the CPU for hundreds of cycles.

3. Optimization Levels

Level	Action	Impact
Level 1 (Design)	Changing O(N^2) to O(N \log N).	Massive (Scalability).
Level 2 (Logic)	Adding Memoization or Pruning.	High (Constant reduction).
Level 3 (Low-level)	Loop unrolling, Bitwise tricks.	Minor (Micro-performance).

Stable Web Resources

- Wikipedia: The P vs NP Problem
- Brilliant.org: Complexity Classes Guide

Quick Revision Cheat Sheet

- **P:** Solvable in polynomial time (Good).
- **NP:** Verifiable in polynomial time (Maybe Good?).
- **NP-Complete:** The "Hardest" problems (Intractable).
- **Comparison Sort Bound:** \Omega(N \log N).

- **Space-Time:** Caching results to save CPU time.
- **Memoization:** Top-down DP caching.
- **Bitmasking:** Using integers as "sets" of flags for speed.
- **Pruning:** Deleting search paths that can't succeed.
- **In-Place:** Uses O(1) auxiliary space.